BUDGETBLISS EVENTS

FINAL YEAR PROJECT

GROUP MEMBERS

SABAHAT SIDDIQUI	BSE213095
BILAL ASHIQ	BSE213113
ASHAR AMJAD	BSE213156

SUPERVISED BY: DR FARAH HANEEF

Spring 2025

Department of Software Engineering

Capital University of Science & Technology, Islamabad

Project Report

VERSION	V X.0

NUMBER OF MEMBERS	03

TITLE	Budgetbliss Events

SUPERVISOR NAME	Dr. Farah Haneef.

MEMBER NAME	REG. NO.	EMAIL ADDRESS
Bilal Ashiq	BSE213113	BSE213113@cust.pk
Sabahat Siddiqui	BSE213095	BSE213095@cust.pk
Ashar Amjad	BSE213156	BSE213156@cust.pk

MEMBERS' SIGNATURES

Bilal Ashiq

Ashar Amjad

Sabahat Siddiqui

Farah Haneef.

Supervisor's Signature

Approval Certificate

This project, entitled as "BudgetBliss Events" has been approved for the award of

Bachelors of Science in Software Engineering

Committee Signatures:

Supervisor: Dr. Farah Haneef. Farah Haneef.

(Supervisor Name Dr / Mr / Ms)

Project Coordinator: _____

(Mr. Ibrar Arshad)

Head of Department: _____

(Dr. Nadeem Anjum)

Declaration

I/We, hereby, declare that "No portion of the work referred to, in this project has been submitted in support of an application for another degree or qualification of this or any other university/institute or other institution of learning". It is further declared thatthis undergraduate project, neither as a whole nor as a part thereof has been copied out from any sources, wherever references have been provided.

MEMBERS' SIGNATURES	
Bilal Ashiq	
Ashar Amjad	
Sabahat Siddiqui	

Acknowledgement

A special note of appreciation goes toward Dr. Farah Haneef, our supervisor, who helped us through this project with her guidance and supervision. Also, appreciation goes toward the Department of Software Engineering, CUST, Islamabad, for facilitating us with the required tools and surroundings necessary for our work.

With gratitude, we also acknowledge family, friends, and colleagues whose constructive criticism assisted in improving BudgetBliss Events. We are thankful to the Android Studio, Firebase, and Google Maps API developers for the provided services which aided in successfully completing our project.

Our hope is that BudgetBliss Events will be advantageous for users and vendors. Additionally, the experience we have gained while completing this project has been incredibly fulfilling.

Executive Summary

BudgetBliss Events is an **advanced event planning platform** that makes organizing weddings, birthdays, and even corporate functions easier. In traditional planning, the vendor bookings and prices are often set in stone which is quite stressful. With BudgetBliss, these issues are all in the past. We solve this with **real-time price negotiations, recommendations, and budget management**— all in one place.

Dynamic pricing, vendor comparisons, and advanced review systems are just a few key features. Built with Android Studio, Firebase," multi-user login sessions", and Google Maps for location services, it ensures no gaps in security and user experience. Under **Dr. Farah Haneef from Capital University of Science & Technology**, the platform was built by a student team and tested exhaustively for reliability.

What makes BudgetBliss stand out among the competitors are flexible pricing, unmatched scalability, and an intuitive interface. Basically, anyone who wants to plan a highly customizable event on a budget can resort to our services, including businesses. New categories of services and new types of services are in the works.

The platform blends smart tech with advanced ergonomic theory at a level never seen before which transforms event planning. It reduces stress and time resulting in incredible cost savings while adding value to it.

Table of Contents

List of Figures

LIST OF TABLES

Chapter 1

1. Introduction

Planning events like **weddings, birthdays and corporate functions** is costly and stressful **[1].** Separately booking a multitude of vendors, from photographers to caterers, consumes a significant amount of time as vendors have set pricing which is inflexible with budgeting **[2].** To simplify matters, **Budget Bliss Events** has developed an all-encompassing hardware and software solution that allows users to service compare and directly **negotiate in real time.** Dynamic user negotiations are made more efficient with the platform's bidding capability, while users benefit from category-based filtering and vendor comparisons to find services tailored to their needs. The platform gathers user preferences through profile customization and activity history to offer a more intuitive planning experience. With features like real-time communication, budget tracking tools, and vendor availability calendars, BudgetBliss enables users to discover suitable vendors and event themes more easily, ensuring an organized and user-centric planning process.

1.1. Project Introduction

An **all-in-one platform for event organization, BudgetBliss Events** caters to weddings, birthday parties, and corporate functions, amongst other life occasions. The BudgetBliss Events evolves around user convenience; unlike **Traditonal Stype platforms** with fixed pricing like Planners Lounge, we allow near perfect **real time negotiations** between users and vendors. (This all but settles the question of using an incorrect term like bidding—this is very much a negotiation system; users and vendors trade offers for which they both accept some price.) This allows users to negotiate value-for-money services. After registering, the user will gain access to unlimited deals and services which allow them to strategize and determine which options suit best for them whilst keeping track of overall expenditures.

In addition, the platform offers enhanced filtering options that help users find services and themes aligned with their preferences such as event type, location, and budget. Users can explore event styles based on categories like traditional, modern, or themed events, making the planning process more intuitive. As users interact with different services and vendors, the platform gradually adapts to present more relevant suggestions through improved search results and organized vendor listings. This allows users to efficiently plan their events with reduced effort and increased personalization..

The previously termed Bidding Systems have been changed to **Real Time Price Negotiation systems** permitting the user to negotiate prices with vendors. The process works in the following sequence:

- Vendors determine the base price for their services.
- Subsequently, users can submit proposals (for example, in a specific acceptable format starting at not less than 80% of the base price).
- Offer, rejection or acceptance is provided by vendors in real time.

Feedback from users and vendors is exchanged instantaneously which enables **BudgetBliss** to ensure that negotiations and communication is undertaken on a rapid basis. Vendors and users use a communication technology known as **WebSockets** which ensures that both users and vendors are updated instantly. For instance, a user posting on the platform must receive seller attention simultaneously and, through effective communication, both can instantaneously information while the two can proactively enables instantaneous provide effective communication easily.WebSockets takes care of this through absolute **real-time connection** wherein there is no need for page reloads. In assuring the fast response with monitoring the connection, the use of the countdown timer takes the lead.

Following are some of the key features of BudgetBliss Events:

- **Vendor Management Module:** With the interface allocates them, the vendors are enabled to register, create profiles as well as offer and manage their services inter

activities. Services are auctioned and vendors make transactions on through the platform.

- **User & Vendor Account Management:** Users can manage their accounts and subscriptions, edit their events, and vendors manage their listings and conversations through a dedicated dashboard. This functionalities contributes to coordination from both ends throughout the program.
- **Vendor Rating & Review System:** Users may rate and review vendors post events. This process encourages transparency and enables users to make decisions based on the vendor's track record.
- **Scalability and extensibility:** The constant addition of new vendors and event categories and users makes the platform self-sustaining, maintaining effectiveness and relevancy to user demand.
- **Customized service:** Users tailor service selection to fit their operational needs. The platform will cater to the user's preferences and budget, thus providing a customized experience for the event.
- **Budget management tools:** These tools assist users in organizing their expenses and budgets throughout the programming. The platform seeks to maintain budgetary limits by suggesting appropriate budget services.
- **Direct interaction with vendors:** The users and vendors can easily communicate and interact without delay. With the use of real-time communication technology (WebSockets), the communication is fast, seamless, and instant leading to easier scheduling and conversations.

To sum it up, BudgetBliss Events simplifies the process of event planning, making it affordable and customized to the user's preferences, thus saving time and effort in connecting them with the right vendors.

1.2. Existing Examples / Solutions

There are some event management platforms available, but most focus on fixed-price services without offering real-time negotiation. Below are some of the existing solutions compared to BudgetBliss Events:

Table 1-1 COMPARISON TO BUDGETBLISS

COMPANIES	DIGITAL PLATFORMS	PHOTOGRAPHER	DECOR	SCHEDULING	BIDDING	MULTI VENDORS	RECOMMENDATIONS
DAWAT	Website	✓	✓	×	×	×	×
EVENTO	Website	×	✓	✓	×	×	×
THE BID DAY	App	✓	✓	✓	×	×	×
BRIDE BOOK	App	✓	✓	✓	×	×	×
MY WED	App	×	✓	✓	×	×	×
WEDWISE	App	✓	✓	✓	×	×	×
ONLINE HALL RESERVATION	Website	×	×	×	×	×	×
BUDGETBLISS EVENTS	App	✓	✓	✓	✓	✓	✓

1.3. Business Scope

In the business segment, **BudgetBliss Events** has strength on a huge and unfulfilled subset of population that individual or company need a tailor-made event planning. It is versatile and scalable: the platform can handle different types of events (birthday, wedding, corporate function, private/public parties, etc.). For realistic pricing and flexibility for the clients,a **dynamic bidding system** helps BudgetBliss Events to change the game of event planning.

Framing the platform as appropriate for **low and high-cost events** presents the listed prices as commercially viable across a wide range of the market.

- **Target Market:** Individuals and businesses seeking to organize private events. Acts as a general market – great for individuals and businesses.
- **Boosted Engagement:** It is likely personalized aspects will improve user engagement and therefore vendor bookings.
- **Business Potential:** The business potential is extensive, due to significant unmet market needs in the area of flexible and low cost pricing event planning services.
- **Scalability:** The platform is easily scalable to add more service categories or types of events to be offered on the platform like conferences, corporate retreats, community events, etc.

1.4. Useful Tools and Technologies

Development Framework and Languages:

- Cross-platform mobile application-development Andriod Studio
- Front-end Dart
- Back-end Node.js

Database:

Realtime data synchronization and storage Firebase Firestore

Cloud Services and Operating System:

The backend hosting was ensured through Google Cloud so that it would be scalable and highly available. Windows was used as the operating system in which the environment of development for ease of setup and support through the IDE.

Security:

Encrypted communication in data transactions - use of HTTPS

Documentation:

- Microsoft Word for project documentation
- Google Docs for real-time collaboration
- LaTex for advanced layouts

Project Management and Communication:

Jira for task management.

Design:

Figma or Adobe XD for UI/UX design and prototyping.

Version Control and Documentation Tools:

GitHub for code versioning and collaboration.

1.5. Project Work Break Down

A WBS or Work Breakdown Structure of BudgetBliss Events helps in dividing the ongoing project into small manageable subtasks which can be worked on in parallel. WBS focuses on project time-line, project cost budgeting and work assignment - breakdown of project into smaller tasks,. Here is WBS includes: Planning & Requirements, Backend Development, Frontend Development, and Testing & Deployment.Every software project is split into segments as represented in figure 1-1.

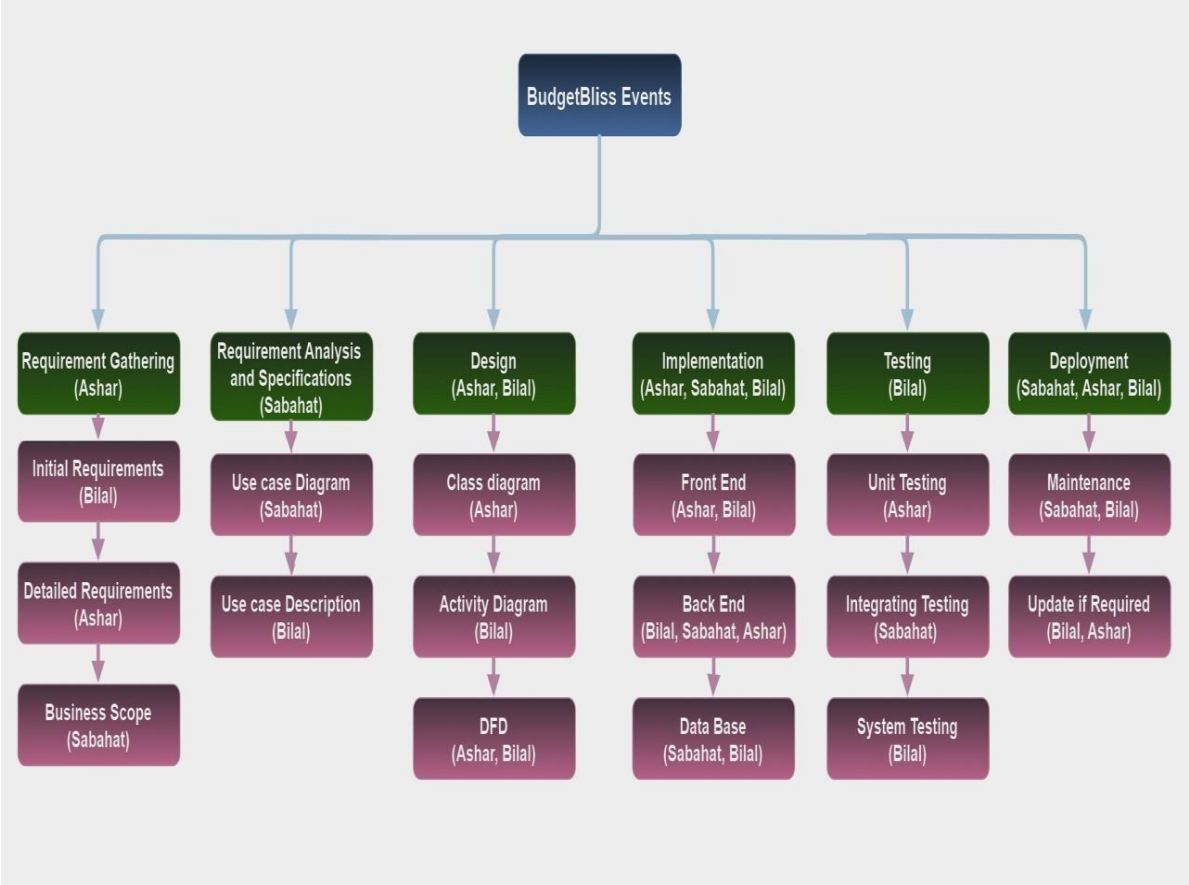


Figure 1-1 Work breakdown structure

1.6. Project Time Line

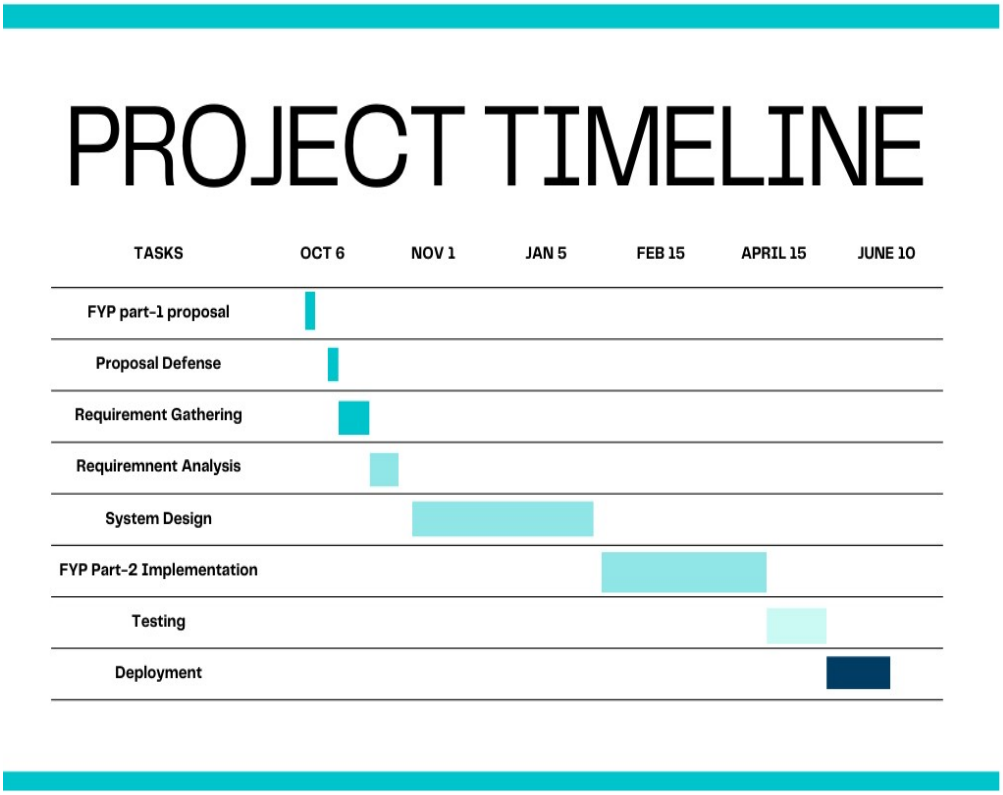


Figure 1-2 Gant Chart

Chapter 2

2. Requirement Specification and Analysis

2.1. Functional Requirements

Table 2- 1 Functional Requirements

S. No.	Functional Requirement	Type	Status
1.	Users (both event planners and vendors) must be able to securely create accounts and log in.	Core	Active

2.	Vendors must have the ability to register, create profiles, and manage their service listings and offers.	Core	Active
3.	Users should be able to update their profile information, including preferences for events.	Core	Active
4.	Users must be able to search for services using filters like location, budget, event type, and vendor ratings.	Core	Active
5.	The system should enable real-time price negotiations between users and vendors through interactive offers and responses.	Core	Active
6.	The system should allow filtering vendors by category, budget, and location	Core	Active
7.	Users should be able to provide feedback on vendors after events, including star ratings.	Productive	Active
8.	Users must have a centralized dashboard to manage their events, track expenses, and handle vendor interactions.	Core	Active
9.	Users and vendors should be able to communicate seamlessly through real-time chat functionality during the negotiation and planning process.	Core	Active
10.	Users must be able to schedule event-related tasks, and the system should send reminders and notifications for upcoming activities.	Productive	Active
11.	The platform should confirm bookings made through the negotiation tool and send confirmation details to both users and vendors.	Core	Active

12.	Users should be able to upload and share event-related documents (e.g., contracts, checklists) through their accounts.	Productive	Active
13.	Vendors must be able to update their availability calendar to avoid scheduling conflicts and provide real-time booking updates.	Productive	Active

2.2. Non-Functional Requirements

Table 2- 2 Non-Functional Requirements

S. No.	Non-Functional Requirement	Category
1.	The platform should handle up to 10,000 simultaneous users with a response time of under 2 seconds for real-time communication.	Performance
2.	The system shall be able to adjust in order to support more services and vendors, as well as add new event types when there is an increase in demand.	Scalability
3.	The platform needs to maintain an uptime of 99.9% in order to provide users and vendors with uninterrupted access.	Availability
4.	Implement HTTPS for data encryption and ensure user authentication through secure protocols.	Security
5.	The platform must handle sudden spikes in user activity without any disruption.	Reliability
6.	Code should be modularand well-documented to facilitate updates and maintenance.	Maintainability

7.	The user interface should be intuitive, ensuring users with minimal technical skills can easily navigate and use the platform.	Usability
8.	User data needs to be handled responsibly, which means the system must comply with having data privacy regulations, such as GDPR.	Regulatory Compliance
9.	Ensure a maximum delay of 1 second during real-time negotiation to keep interactions smooth and efficient.	Performance
10.	Platform must support automatic backups with data recovery within 2 hours of failure.	Reliability

2.3. System Use Case Modeling

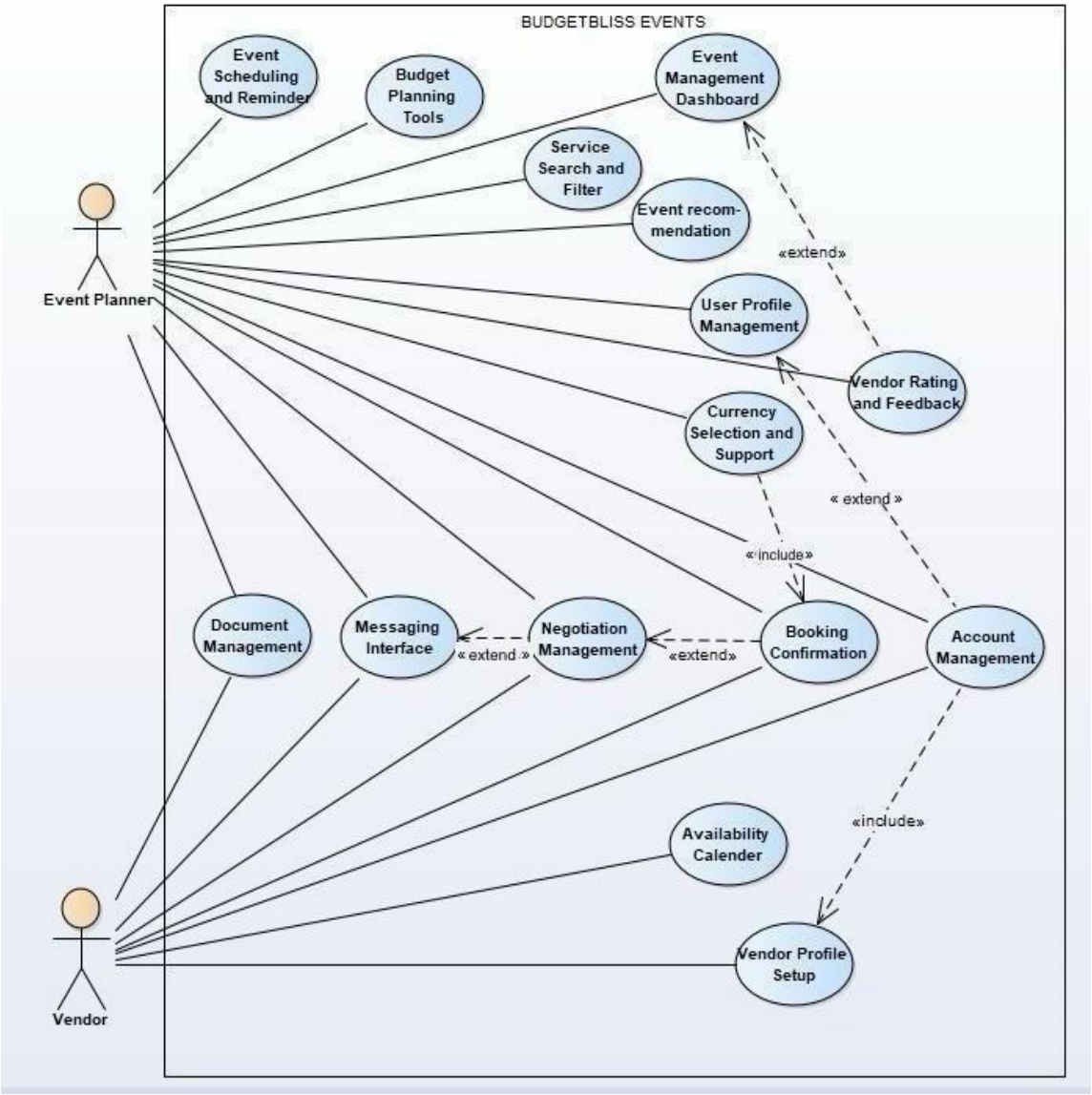


Figure2- 1 Use Case Diagram

2.3.1 Use Case 1 (User Registration and Authentication)

Table 2-3- 1 User Registration and Authentication

Use Case ID:	UC-01		
Use Case Name:	User Registration and Authentication		
Created By:	Sabahat Siddiqui	**Last Updated By:**	Sabahat Siddiqui
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User (Event Planner, Vendor)		
Description:	Allows users to securely register and log in to access the platform.		
Trigger:	A new user registers or an existing user logs in.		
Preconditions:	User has a valid email address and password.		
Post conditions:	User is authenticated and logged in.		
Normal Flow:	**Actor**	**System**	
	1. Selects "Sign Up" or "Log In". 2. Provides credentials.	1. Prompts for email and password. 2. Validates credentials and grants access.	
Alternative Flows:	User forgets password and resets it.		
Exceptions:	Invalid credentials return an error message.		

2.3.2 Use Case 2 (Vendor Management Module)

Table 2-3- 2 Vendor Management Module

Use Case ID:	UC-02		
Use Case Name:	Vendor Management Module		
Created By:	Sabahat Siddiqui	**Last Updated By:**	Sabahat Siddiqui
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	Vendor		
Description:	Allows vendors to register, create profiles, and manage their listings.		
Trigger:	Vendor wants to create or update a profile.		
Preconditions:	Vendor is registered.		
Post conditions:	Vendor profile and listings are updated.		
Normal Flow:	**Actor**	**System**	
	1. Vendor logs in and accesses profile settings. 2. Vendor updates profile and submits.	1. Displays profile and listing options. 2. Saves changes.	
Alternative Flows:	• Vendor chooses to deactivate their profile temporarily. • Vendor requests assistance from assist to update profile statistics.		
Exceptions:	System errors prompt the vendor to retry.		

2.3.3 Use Case 3 (User Profile Management)

Table 2-3- 3 User Profile Management

Use Case ID:	UC-03		
Use Case Name:	User Profile Management		
Created By:	Sabahat Siddiqui	**Last Updated By:**	Sabahat Siddiqui
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User (Event Planner, Vendor)		
Description:	Enables users to update their profile information and preferences.		
Trigger:	User chooses to update profile settings.		
Preconditions:	User is logged in.		
Post conditions:	Profile is updated with new information.		
Normal Flow:	**Actor**	**System**	
	1. Navigates to profile settings. 2. Updates information and saves changes.	1. Displays profile details. 2. Confirms update.	
Alternative Flows:	• User decides to reset their preferences to default settings. • User uploads a profile picture that fails validation and receives a prompt to retry.		
Exceptions:	System errors during saving.		

2.3.4 Use Case 4 (Search and Filtering System)

Table 2-3- 4 Search and Filtering System

Use Case ID:	UC-04		
Use Case Name:	Search and Filtering System		
Created By:	Sabahat Siddiqui	**Last Updated By:**	Sabahat Siddiqui
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User (Event Planner)		
Description:	Users are able to look up services through keyword searches with filters such as location and budget.		
Trigger:	The user performs a search.		
Preconditions:	The user is already signed in.		
Post conditions:	The system provides the user with the most relevant search options.		
Normal Flow:	**Actor**	**System**	
	1. Enters search criteria.	1. Filters results based on criteria. 2. Displays filtered results.	
Alternative Flows:	• User saves a search query for future reference. • User clears applied filters to restart the search.		
Exceptions:	No results found.		

2.3.5 Use Case 5 (Real-Time Price Negotiation)

Table 2-3- 5 Real-Time Price Negotiation

Use Case ID:	UC-05		
Use Case Name:	Real-Time Price Negotiation		
Created By:	Sabahat Siddiqui	**Last Updated By:**	Sabahat Siddiqui
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User, Vendor		
Description:	Enables real-time price negotiation between users and vendors.		
Trigger:	User initiates negotiation on a vendor listing.		
Preconditions:	Both parties are logged in.		
Post conditions:	Agreement is reached, or negotiation ends.		
Normal Flow:	**Actor**	**System**	
	1. User sends a negotiation request. 2. Vendor reviews and responds.	1. Notifies vendor. 2. Relays responses until agreement.	
Alternative Flows:	User declines offer.		
Exceptions:	Network disruptions.		

2.3.6 Use Case 7 (Budget Management Tools)

Table 2-3- 6 Budget Management Tools

Use Case ID:	UC-07		
Use Case Name:	Budget Management Tools		
Created By:	Ashar Amjad	**Last Updated By:**	Ashar Amjad
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User (Event Planner)		
Description:	Provides tools to set, monitor, and adjust the event budget.		
Trigger:	User accesses budget tools.		
Preconditions:	User is planning an event.		
Post conditions:	Budget is adjusted as needed.		
Normal Flow:	**Actor**	**System**	
	1. Navigates to budget management. 2. Adjusts budget settings.	1. Displays current budget status. 2. Updates budget details.	
Alternative Flows:	• User exports budget data to an external file for offline analysis. • User adjusts budget by adding or removing pre-defined expense categories.		
Exceptions:	Budget limit exceeded.		

2.3.7 Use Case 8 (Vendor Ratings and Review System)

Table 2-3- 7 Vendor Ratings and Review System

Use Case ID:	UC-08		
Use Case Name:	Vendor Ratings and Review System		
Created By:	Ashar Amjad	**Last Updated By:**	Ashar Amjad
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User (Event Planner)		
Description:	Allows users to rate and review vendors after events.		
Trigger:	User submits a review.		
Preconditions:	User has completed an event with the vendor.		
Post conditions:	Review is saved.		
Normal Flow:	**Actor**	**System**	
	1. Submits a review.	1. Saves rating and feedback.	
Alternative Flows:	• User edits a previously submitted review. • User rates the vendor without providing written feedback.		
Exceptions:	Inappropriate review flagged.		

2.3.8 Use Case 9 (Event Planning Dashboard)

Table 2-3- 8 Event Planning Dashboard

Use Case ID:	UC-09		
Use Case Name:	Event Planning Dashboard		
Created By:	Ashar Amjad	**Last Updated By:**	Ashar Amjad
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User (Event Planner)		
Description:	Provides a centralized dashboard for event management.		
Trigger:	User accesses the dashboard.		
Preconditions:	User is planning an event.		
Post conditions:	User views and manages event details.		
Normal Flow:	**Actor**	**System**	
	1. Opens dashboard. 2. Manages tasks and expenses.	1. Displays event information.	
Alternative Flows:	• User customizes the dashboard layout to highlight priority tasks. • User adds collaborators to manage shared tasks.		
Exceptions:	Dashboard data fails to load.		

2.3.9 Use Case 10 (Communication Interface)

Table 2-3- 9 Communication Interface

Use Case ID:	UC-10		
Use Case Name:	Communication Interface		
Created By:	Ashar Amjad	**Last Updated By:**	Ashar Amjad
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User, Vendor		
Description:	Enables real-time chat for negotiations and planning.		
Trigger:	User initiates a chat with a vendor.		
Preconditions:	Both parties are logged in.		
Post conditions:	Communication is established.		
Normal Flow:	**Actor**	**System**	
	1. Starts chat. 2. Exchanges messages.	1. Connects both parties.	
Alternative Flows:	• User archives the chat after negotiation is complete. • User forwards the conversation summary to their email.		
Exceptions:	Network issues disrupt chat.		

2.3.10 Use Case 11 (Event Scheduling and Notifications)

Table 2-3- 10 Event Scheduling and Notifications

Use Case ID:	UC-11		
Use Case Name:	Event Scheduling and Notifications		
Created By:	Bilal Ashiq	**Last Updated By:**	Bilal Ashiq
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User		
Description:	Allows users to schedule event tasks and receive reminders for upcoming activities.		
Trigger:	User schedules a task or an event date.		
Preconditions:	User has an active event with planned tasks.		
Post conditions:	Task or event is scheduled, and notifications are set up.		
Normal Flow:	**Actor**	**System**	
	1. Selects "Add Task" or "Set Reminder". 2. Enters task details and sets a reminder time.	1. Displays task creation form. 2. Validates and saves task, confirms setup.	
Alternative Flows:	Option to adjust reminder settings or reschedule tasks.		
Exceptions:	Displays error if unable to save or schedule the task.		

2.3.11 Use Case 12 (Service Booking Confirmation)

Table 2-3- 11 Service Booking Confirmation

Use Case ID:	UC-12		
Use Case Name:	Service Booking Confirmation		
Created By:	Bilal Ashiq	**Last Updated By:**	Bilal Ashiq
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User, Vendor		
Description:	Confirms bookings made through the negotiation tool and sends confirmation details to both users and vendors.		
Trigger:	User and vendor agree on service terms.		
Preconditions:	User and vendor have completed negotiation.		
Post conditions:	Booking confirmation is sent to both parties.		
Normal Flow:	**Actor**	**System**	
	1. Confirms booking terms and requests confirmation.	1. Displays booking summary for final confirmation.	
Alternative Flows:	Option to modify or reschedule the booking.		
Exceptions:	Displays error if booking confirmation fails.		

2.3.12 Use Case 13 (Document Upload and Sharing)

Table 2-3- 12 Document Upload and Sharing

Use Case ID:	UC-13		
Use Case Name:	Document Upload and Sharing		
Created By:	Bilal Ashiq	**Last Updated By:**	Bilal Ashiq
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	User, Vendor		
Description:	Allows users to upload and share event-related documents (e.g., contracts, checklists).		
Trigger:	User or vendor initiates document upload.		
Preconditions:	User or vendor has a valid account with an active event.		
Post conditions:	Document is uploaded and accessible for viewing.		
Normal Flow:	**Actor**	**System**	
	1. Selects "Upload Document". 2. Uploads document file.	1. Displays file upload prompt. 2. Validates and saves document, confirms upload.	
Alternative Flows:	Option to delete or replace an uploaded document.		
Exceptions:	Displays error if document upload fails.		

2.3.13 Use Case 15 (Vendor Availability Calendar)

Table 2-3- 13 Vendor Availability Calendar

Use Case ID:	UC-15		
Use Case Name:	Vendor Availability Calendar		
Created By:	Bilal Ashiq	**Last Updated By:**	Bilal Ashiq
Date Created:	22/11/24	**Last Revision Date:**	22/11/24
Actors:	Vendor		
Description:	Allows vendors to update their availability calendar and avoid scheduling conflicts.		
Trigger:	Vendor updates their availability status.		
Preconditions:	Vendor has an active profile and services listed.		
Post conditions:	Vendor availability is updated and visible to users.		
Normal Flow:	**Actor** 1. Selects "Update Availability". 2. Marks available or unavailable dates.	**System** 1. Displays calendar for availability updates. 2. Saves availability changes and updates the vendor profile.	
Alternative Flows:	Option to mark dates as "tentative" if needed.		
Exceptions:	Displays error if availability update fails.		

2.4. System Sequence diagrams

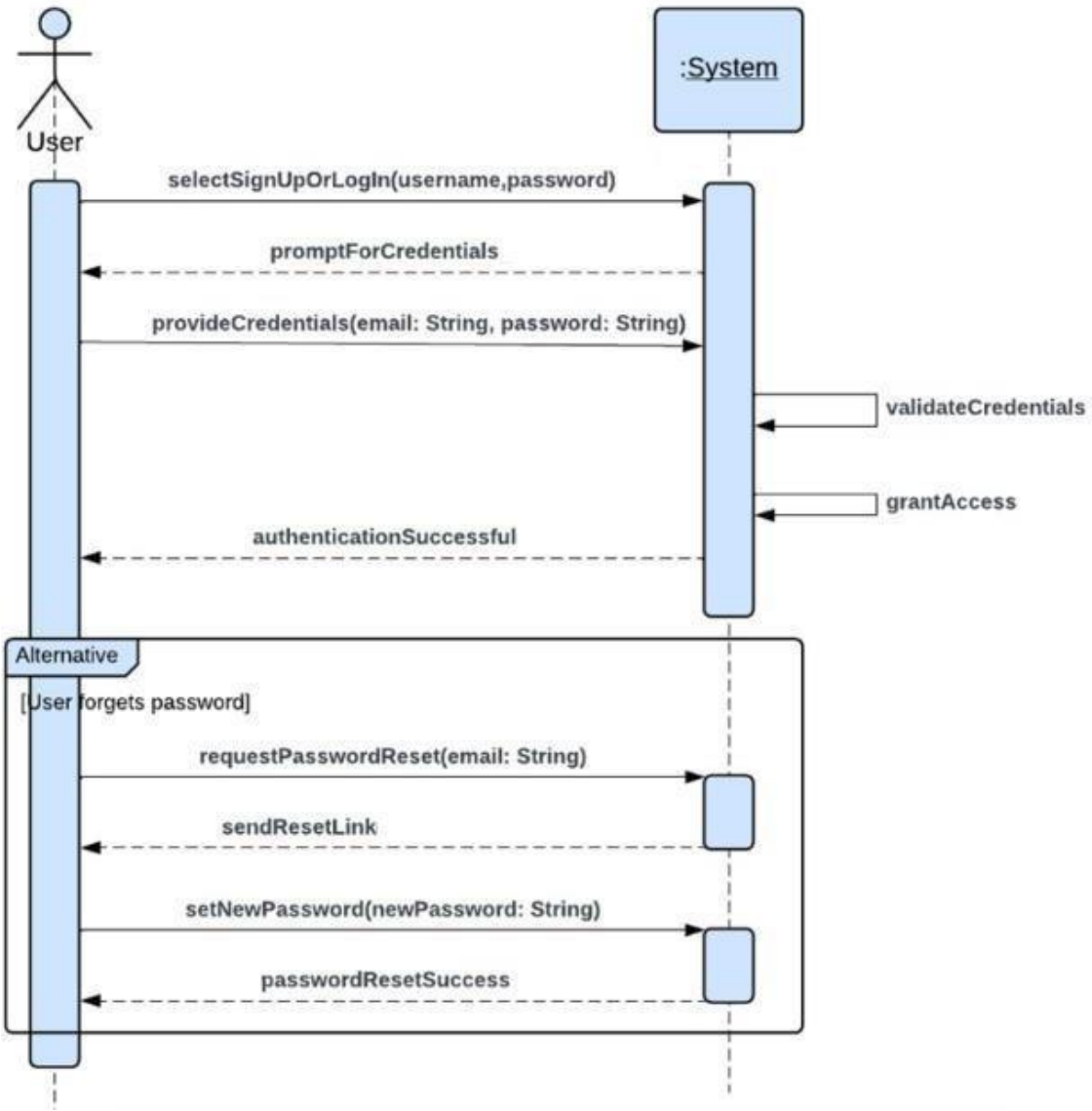


Figure 2-1 User Registration and Authentication

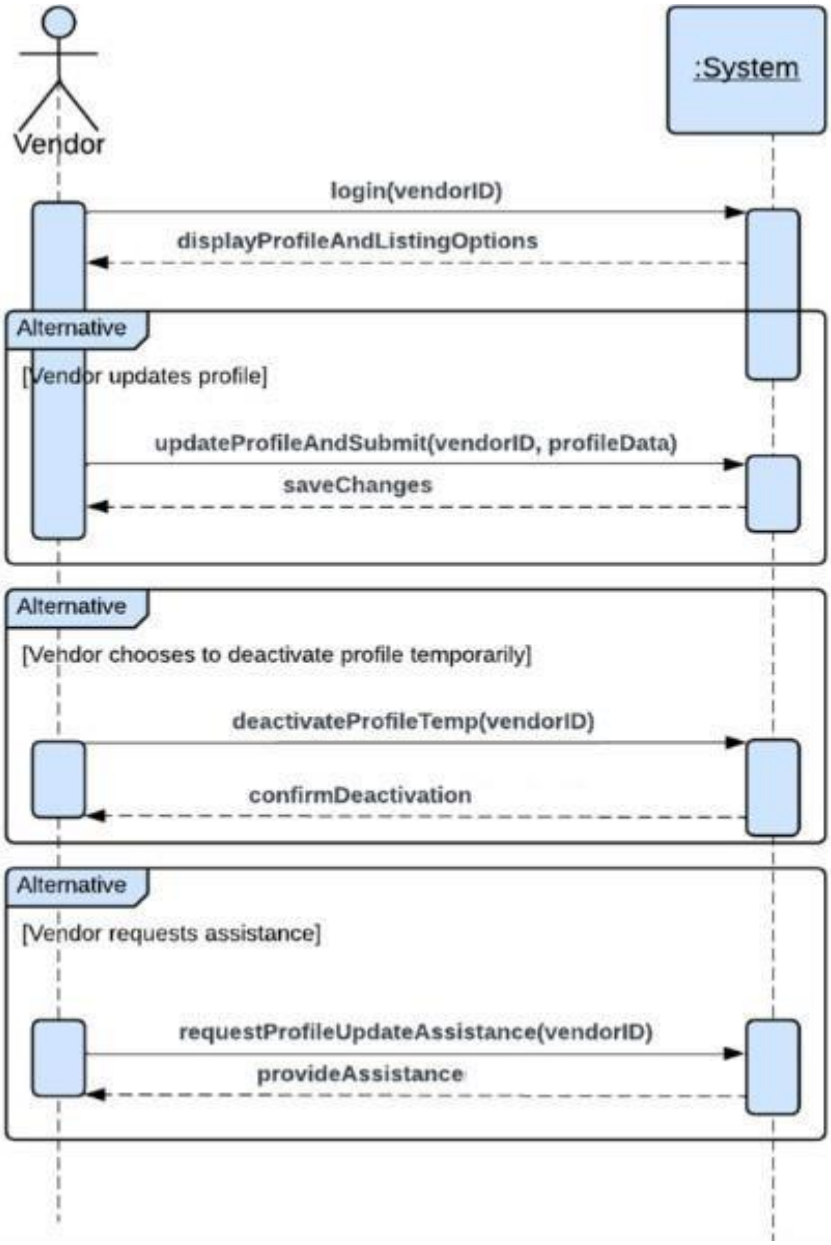


Figure 2-2 Vendor Management Module

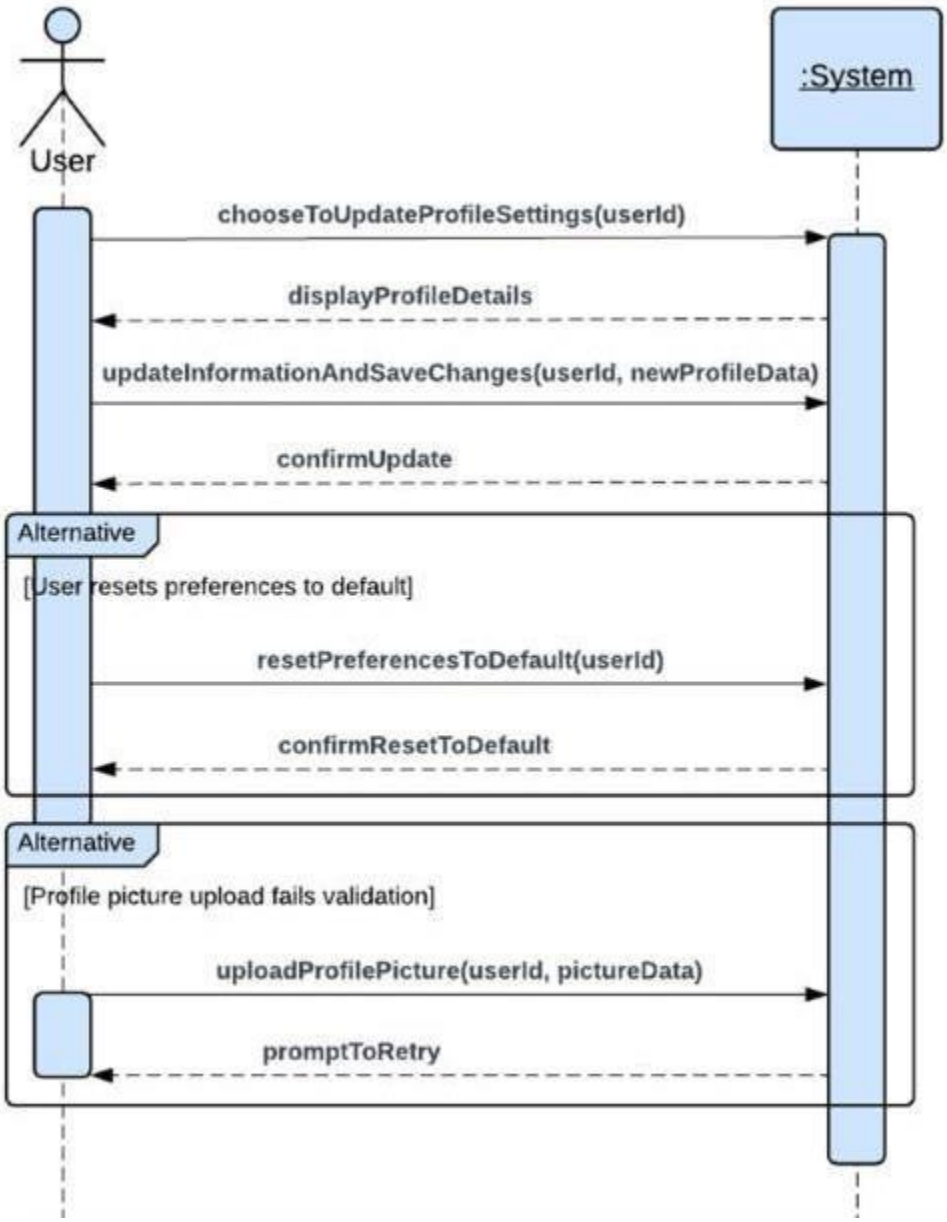


Figure 2-3 User Profile Management

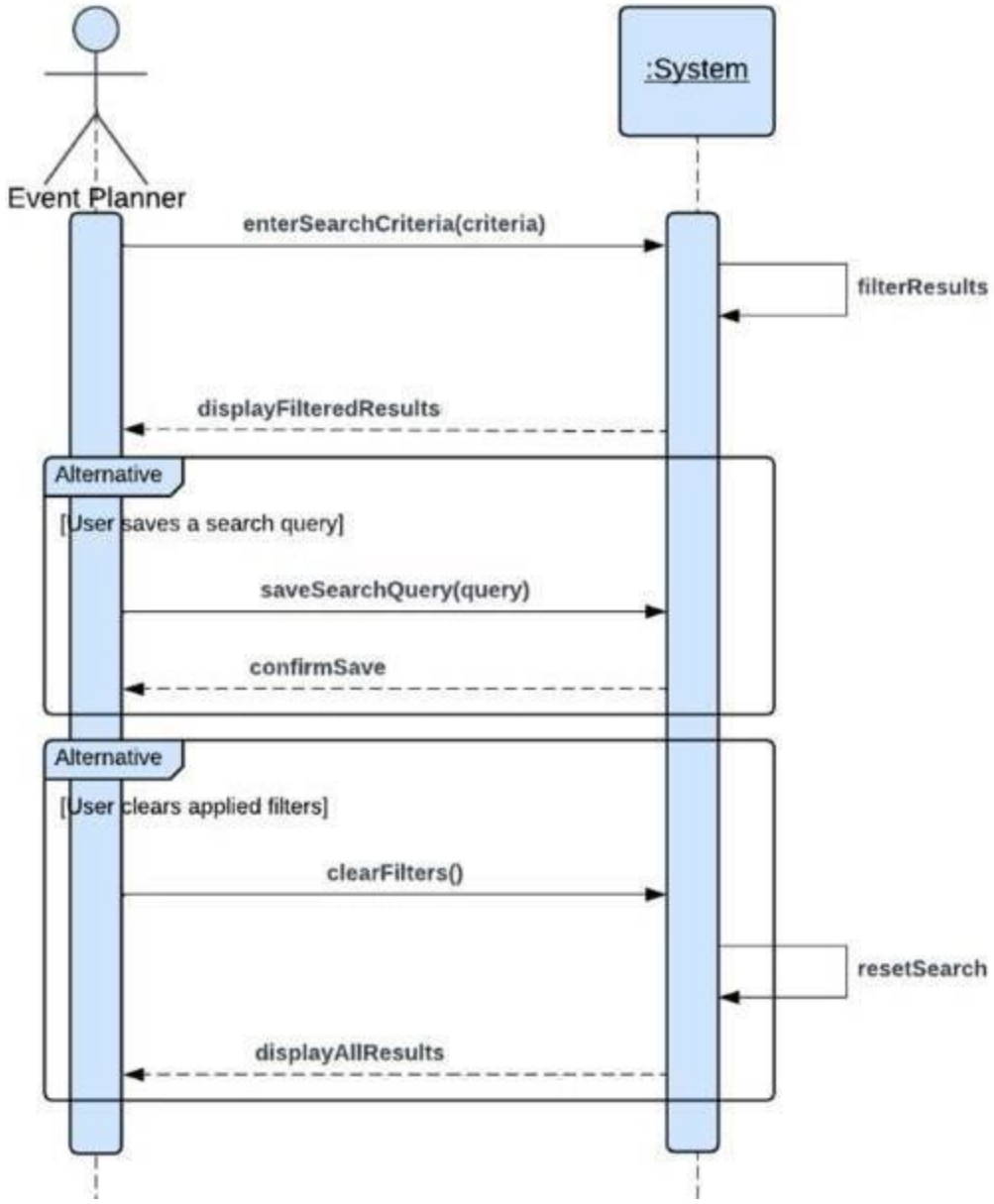


Figure 2-4 Search and Filtering System

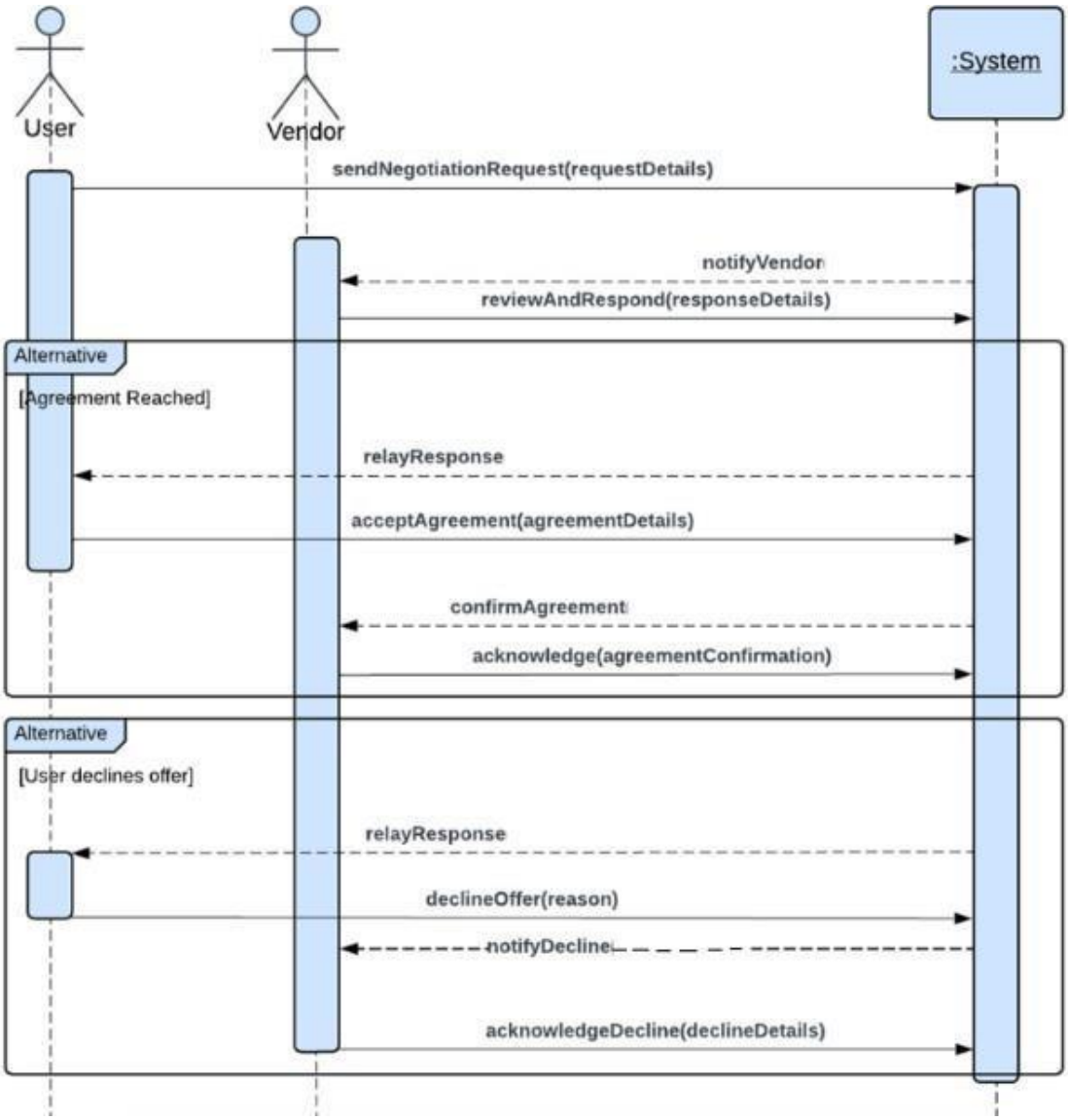


Figure 2-5 Real-Time Price Negotiation

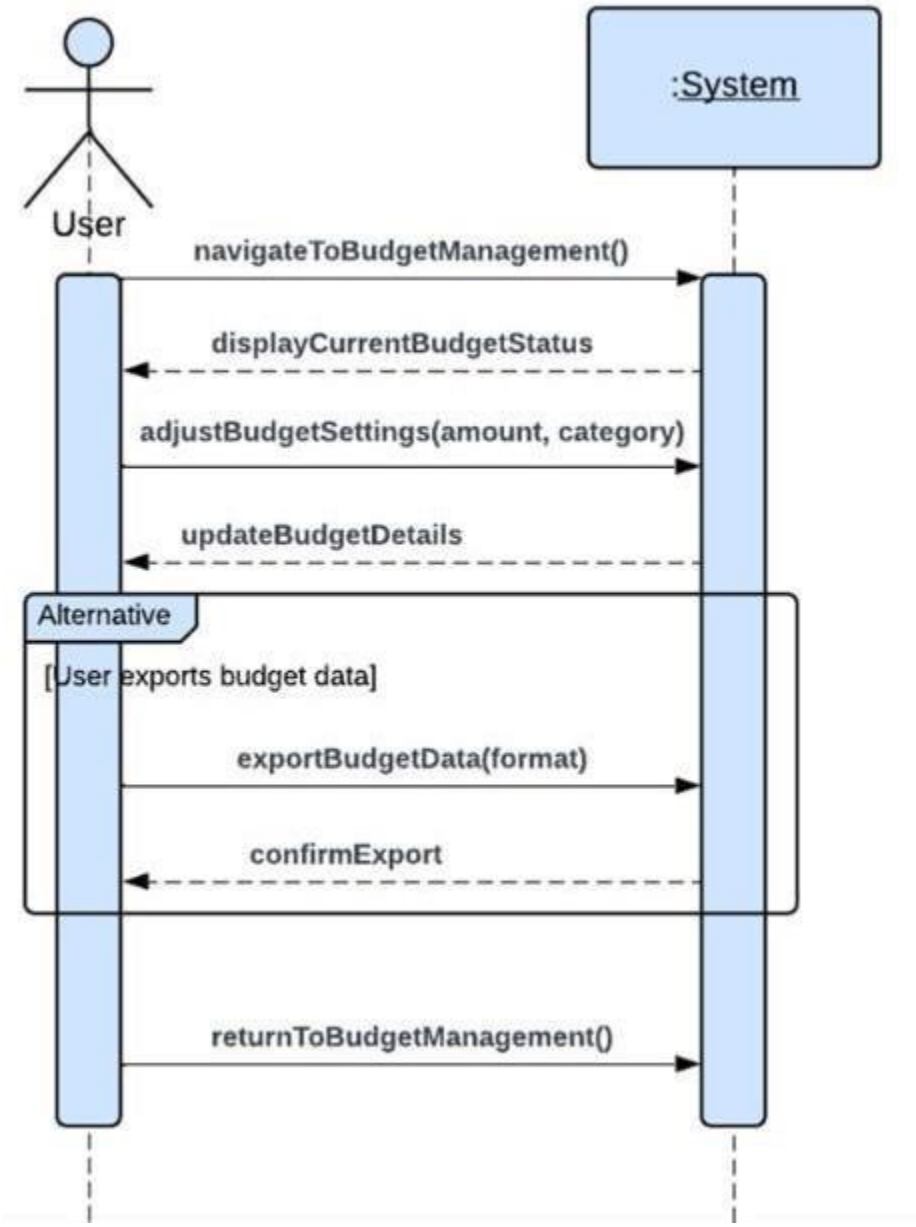


Figure 2-6 Budget Management Tools

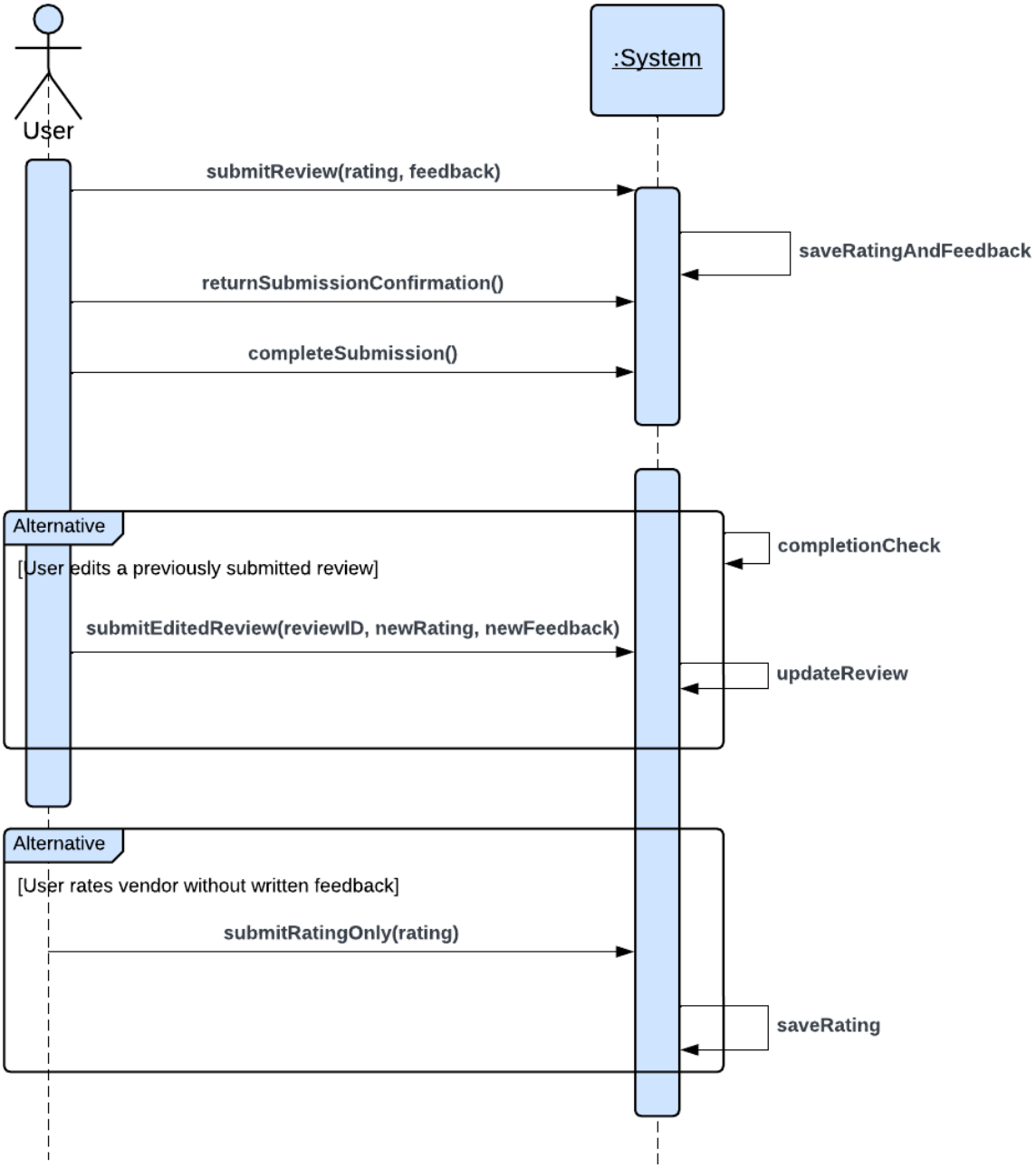


Figure 2-7 Vendor Ratings and Review System

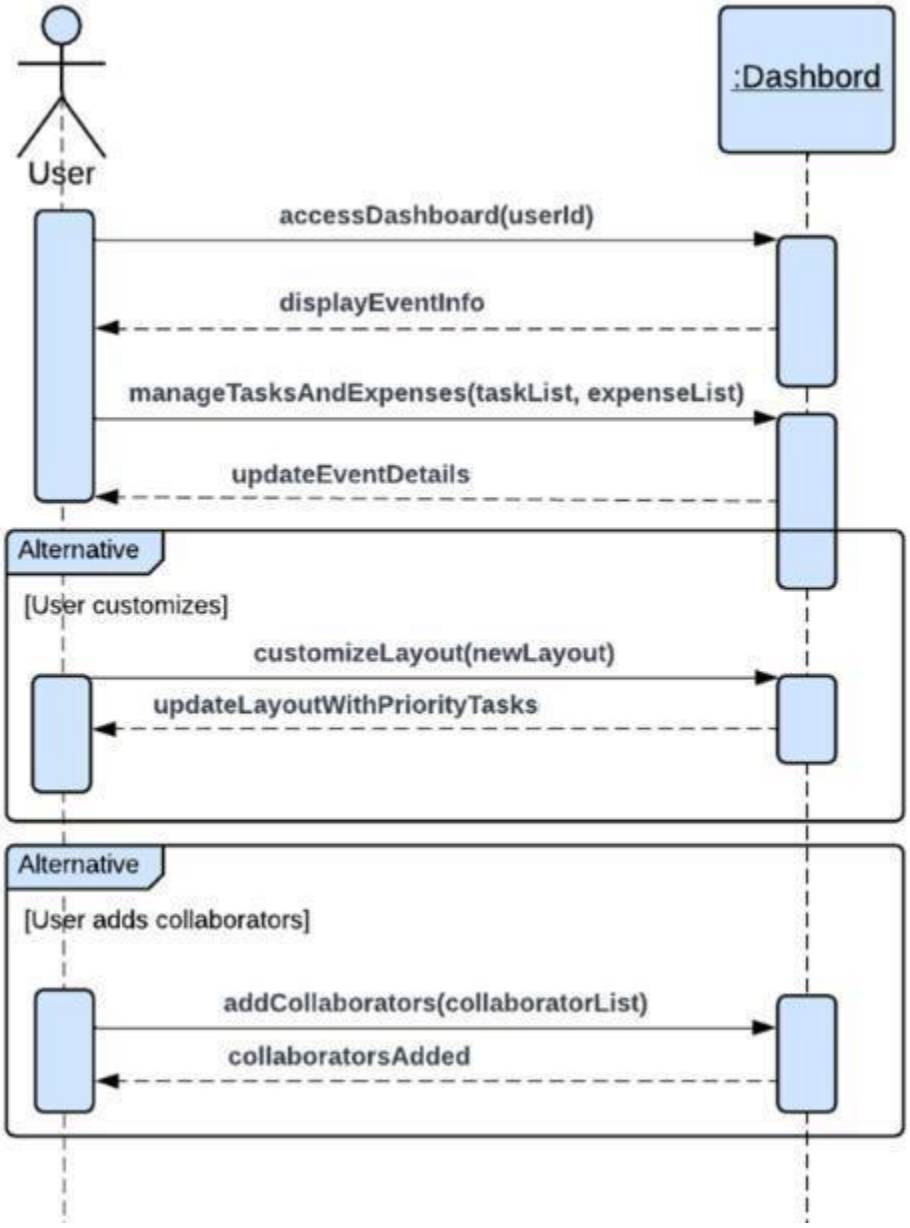


Figure 2-8 Event Planning Dashboard

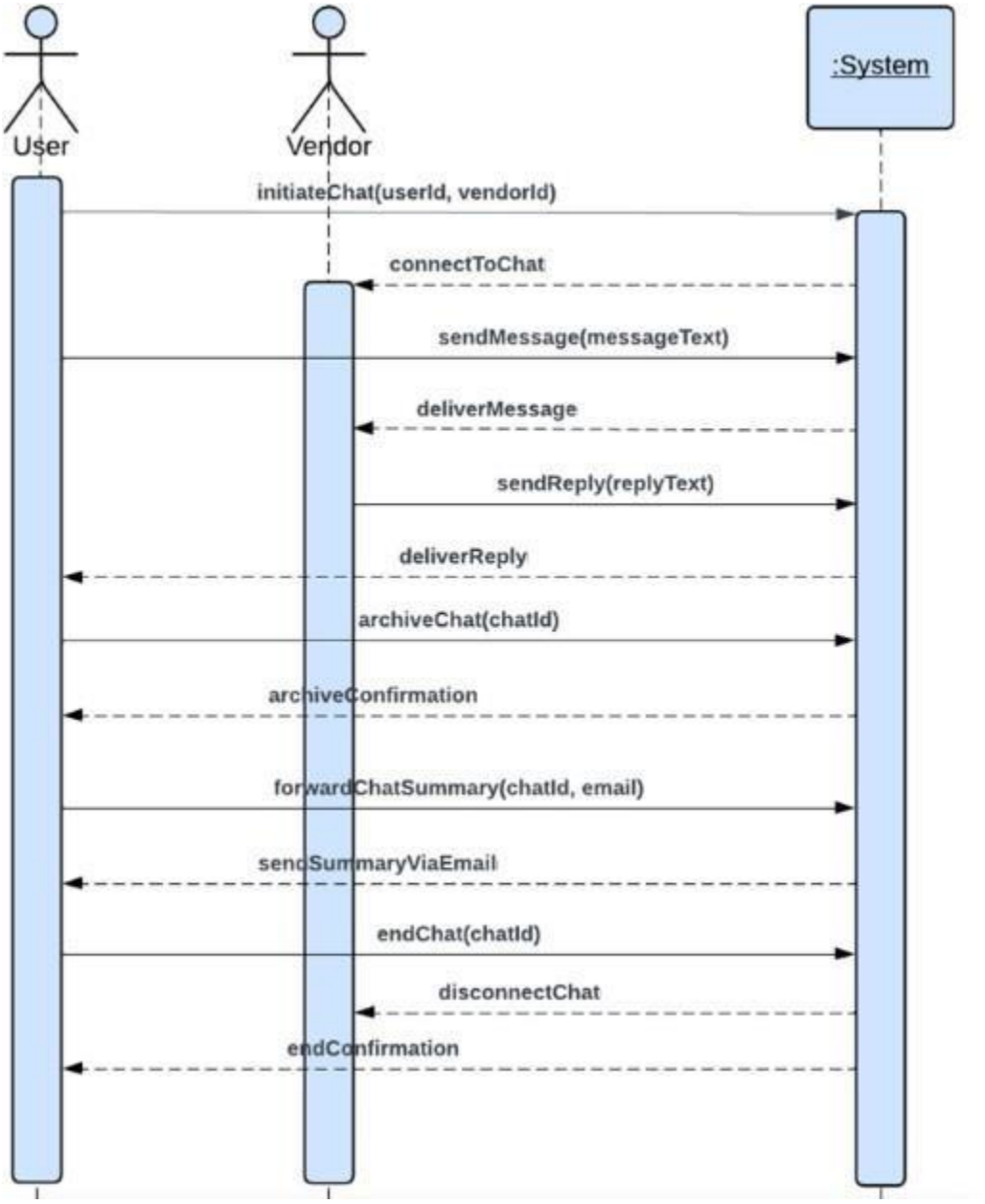


Figure 2-9 Communication Interface

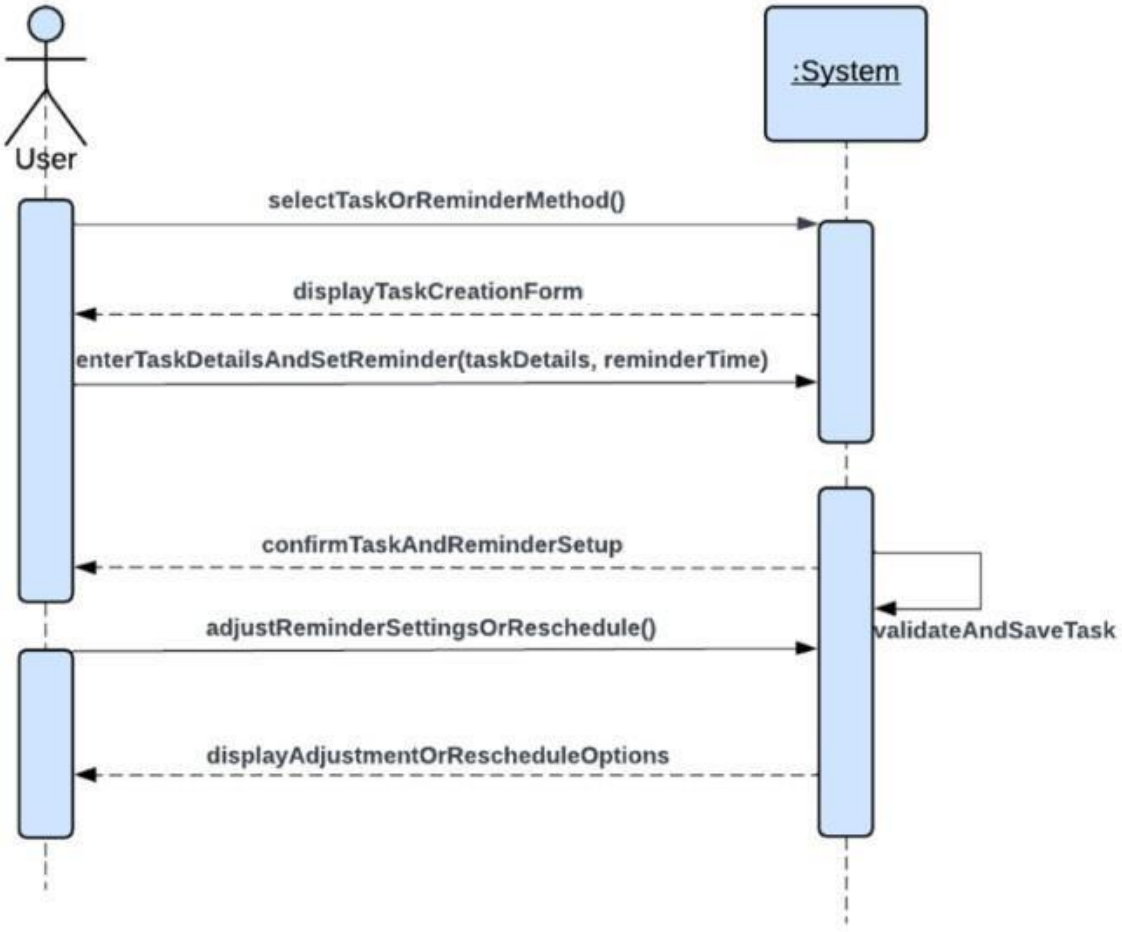


Figure 2-10 Event Scheduling and Notifications

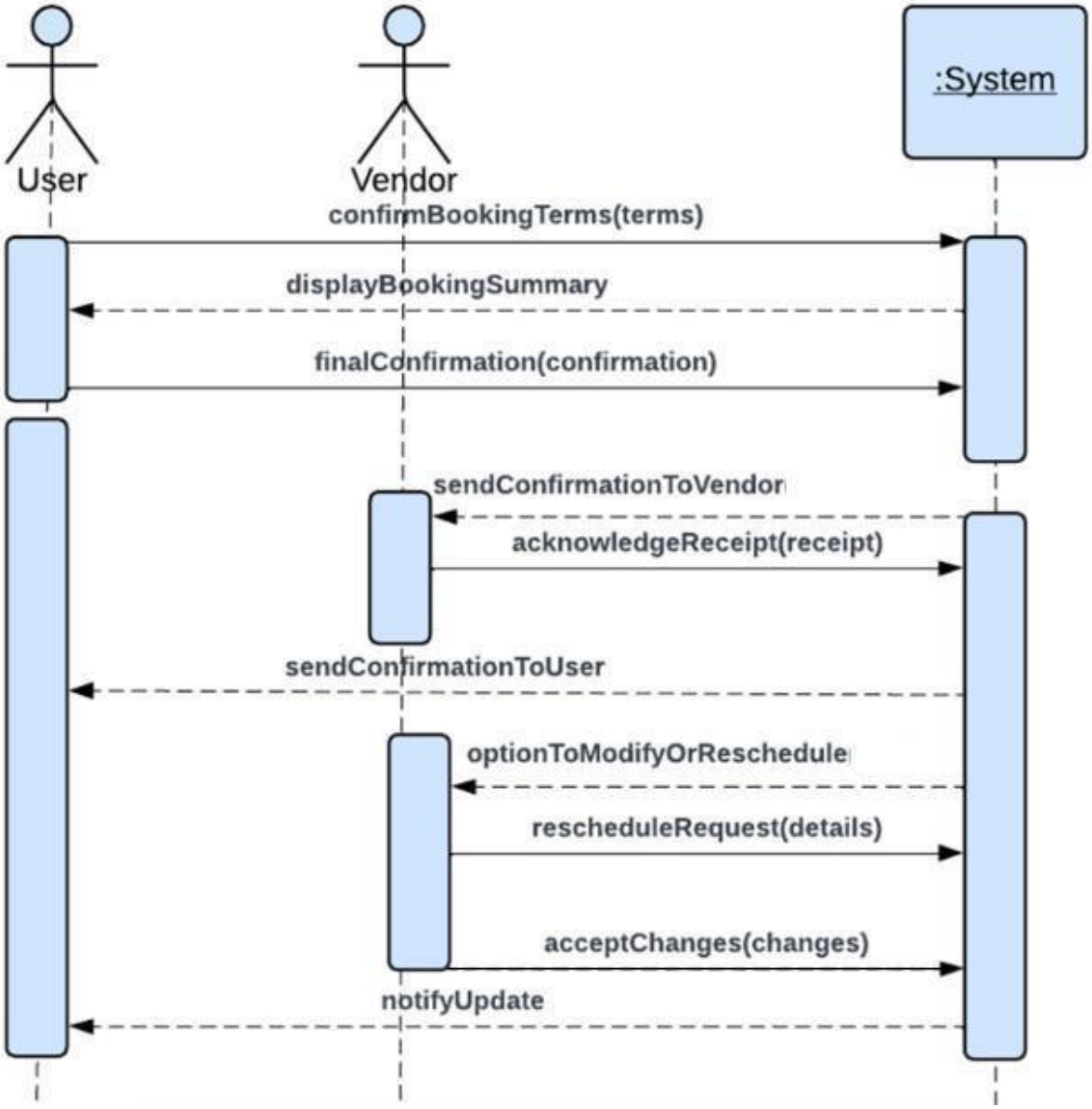


Figure 2-11 Service Booking Confirmation

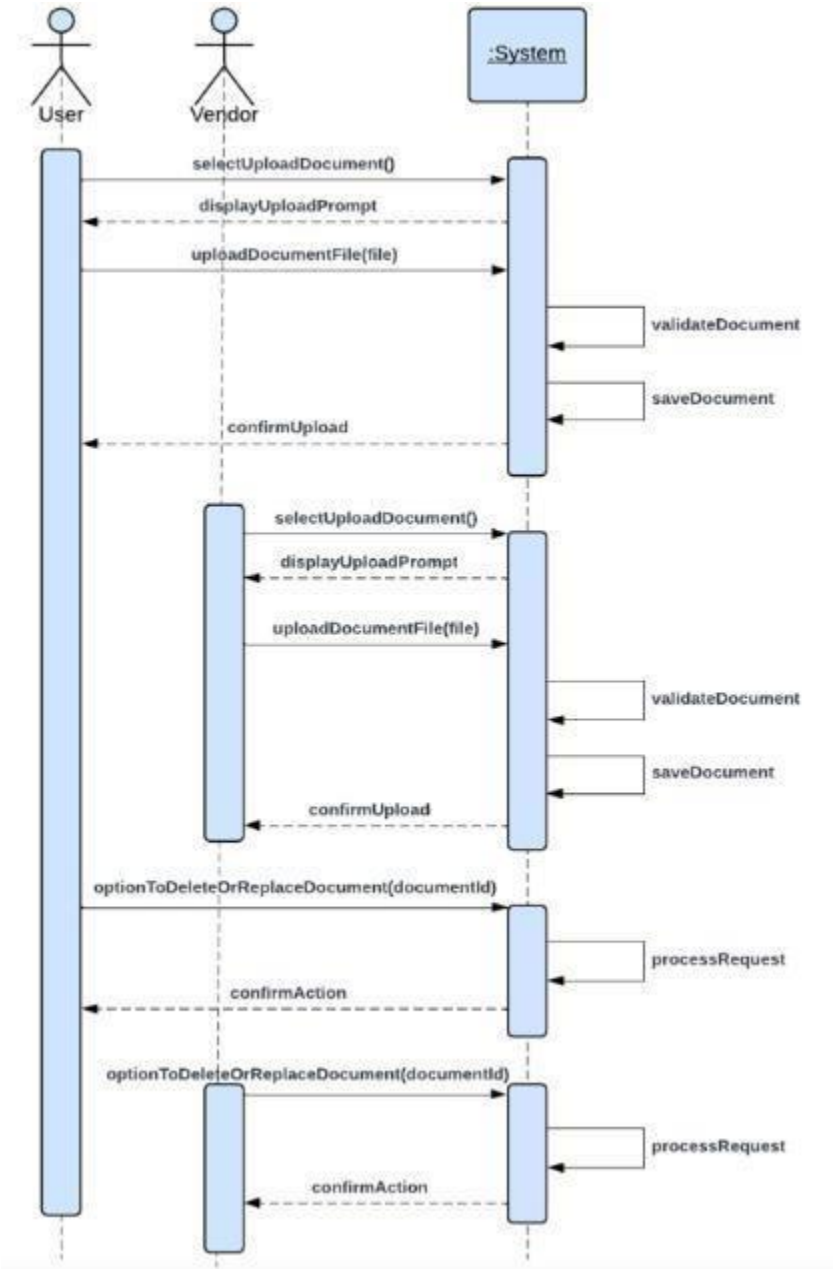


Figure 2-12 Document Upload and Sharing

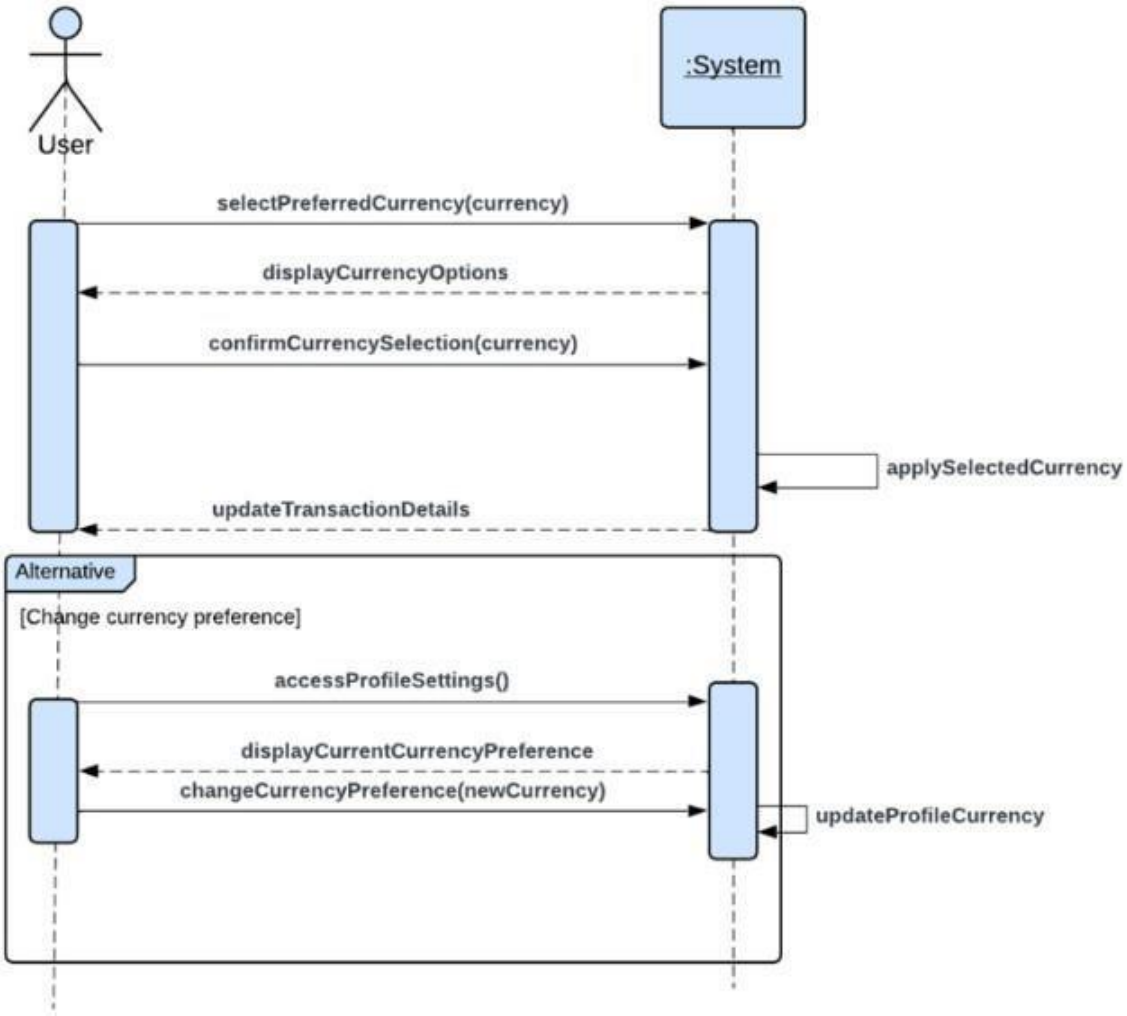


Figure 2-13 Multi-Currency Support

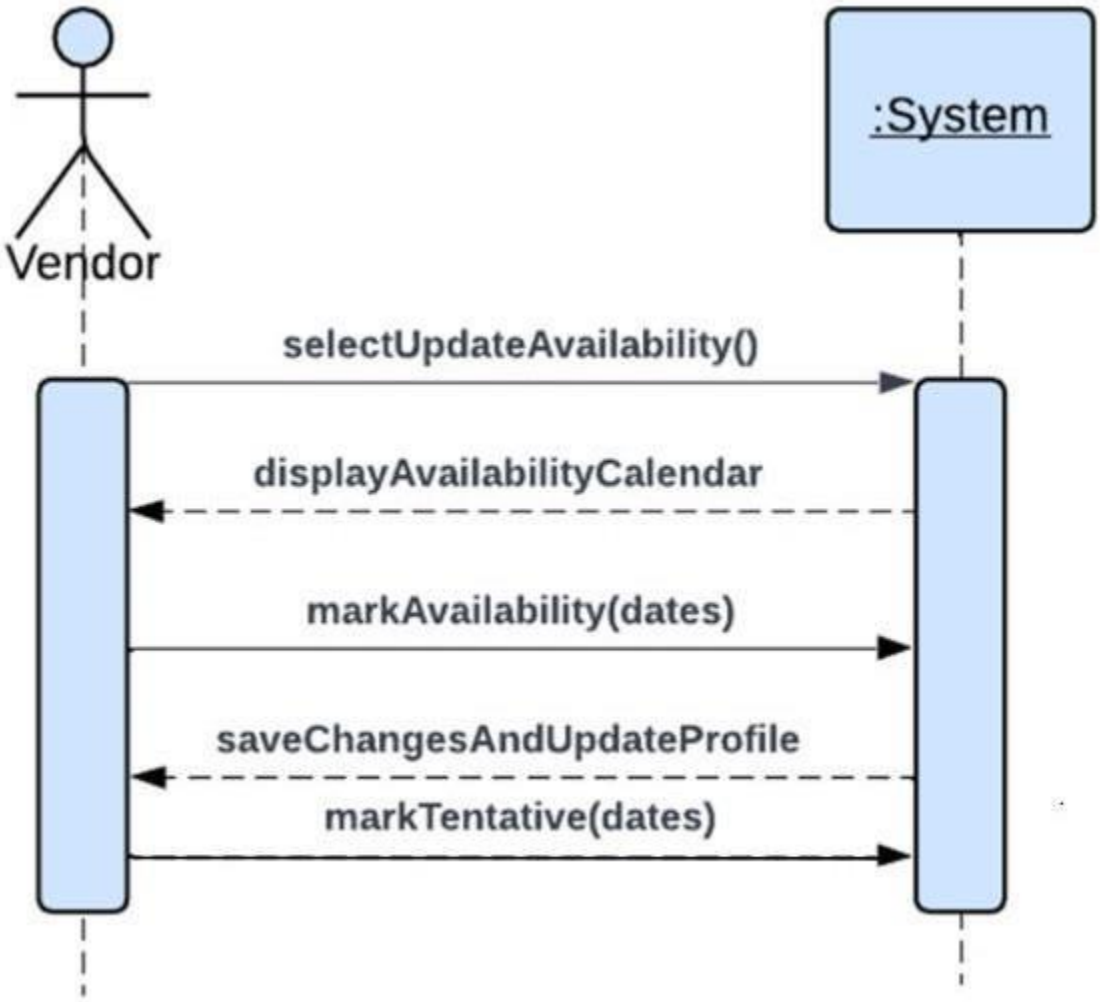


Figure 2-14 Vendor Availability Calendar

2.4 Domain Model

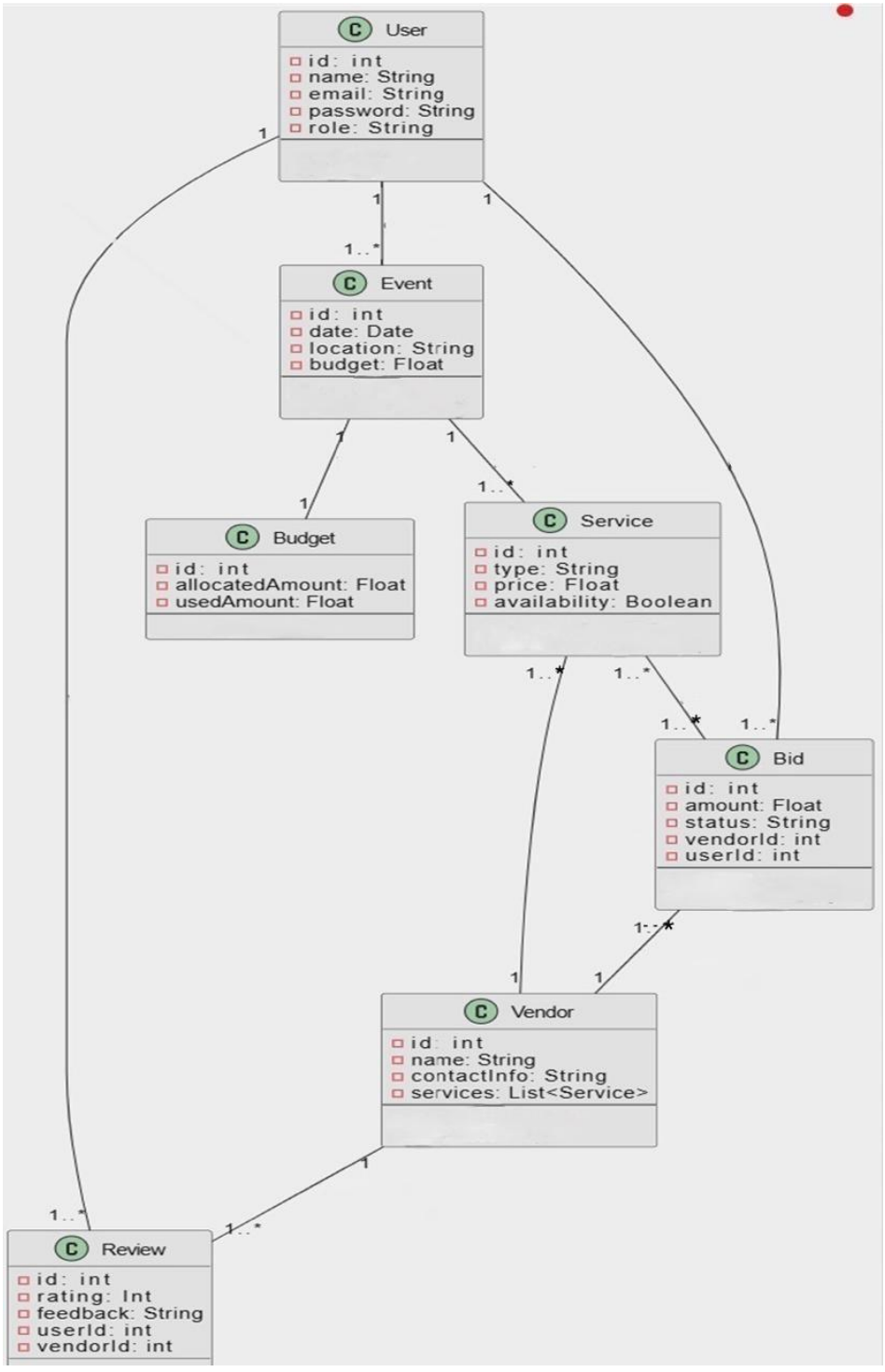


Figure 2-15 Domain Model

2.5 User Interface Design (Prototypes)

Figure 2-16 Home Page

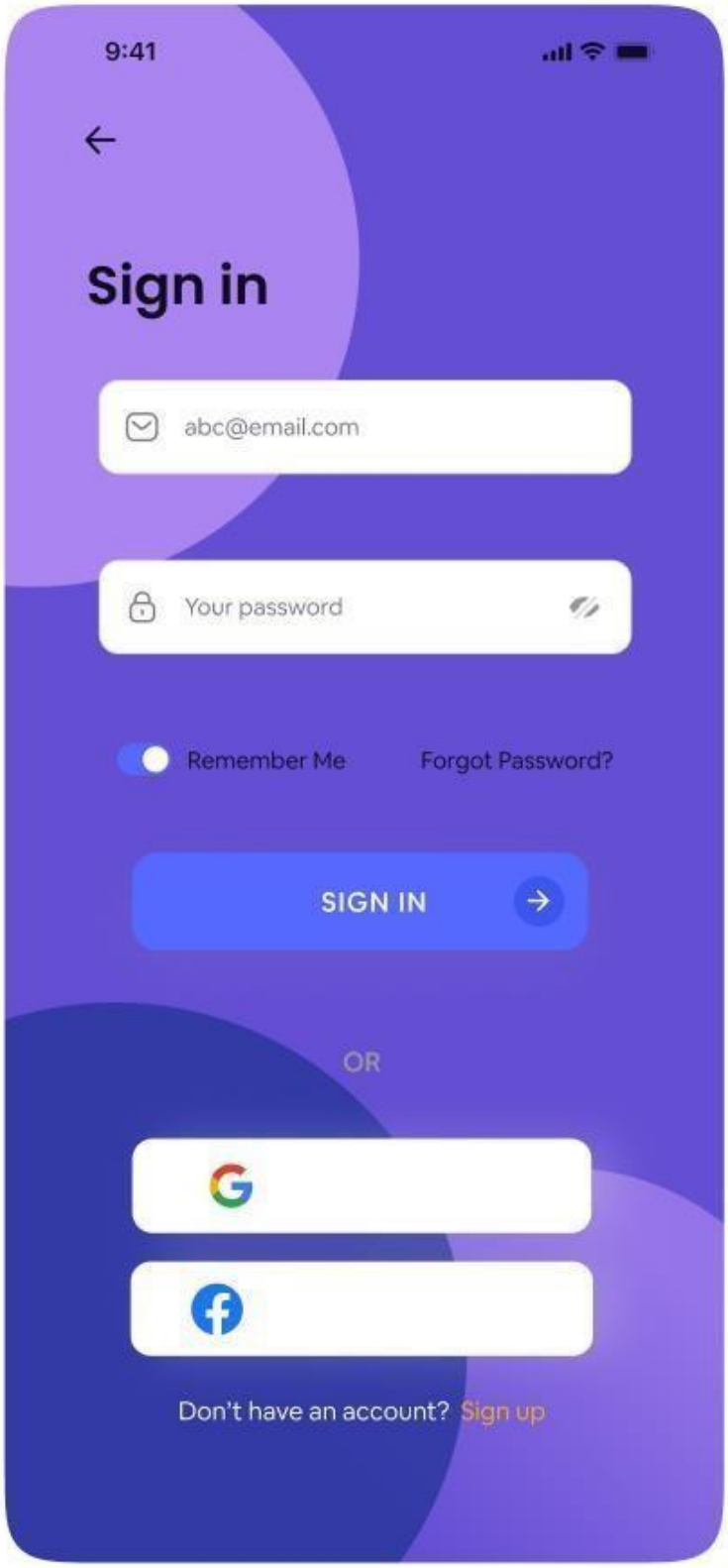


Figure 2-18 Sign in

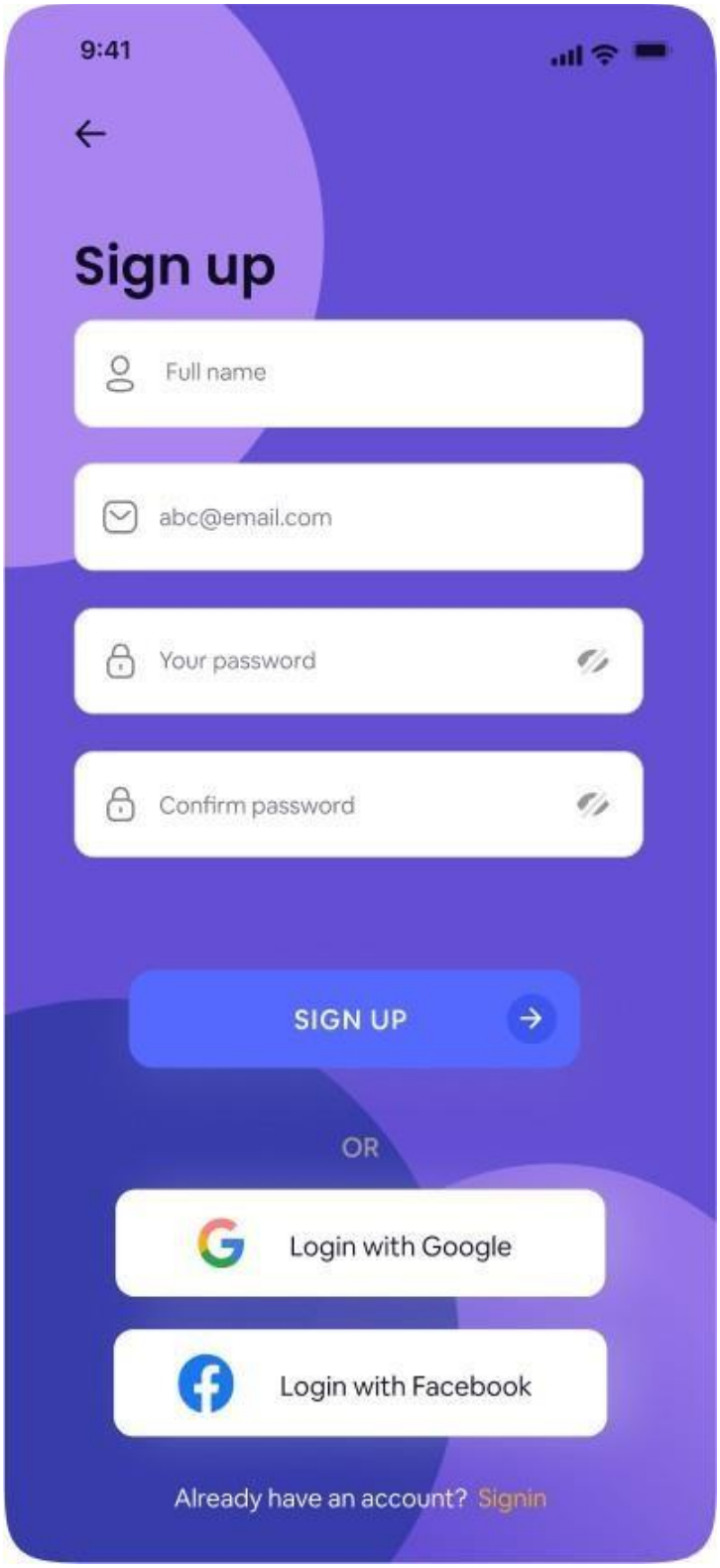


Figure 2-19 Sign Up

Figure 2-20 Event Planner Home Page

Figure 2-21 Create Event

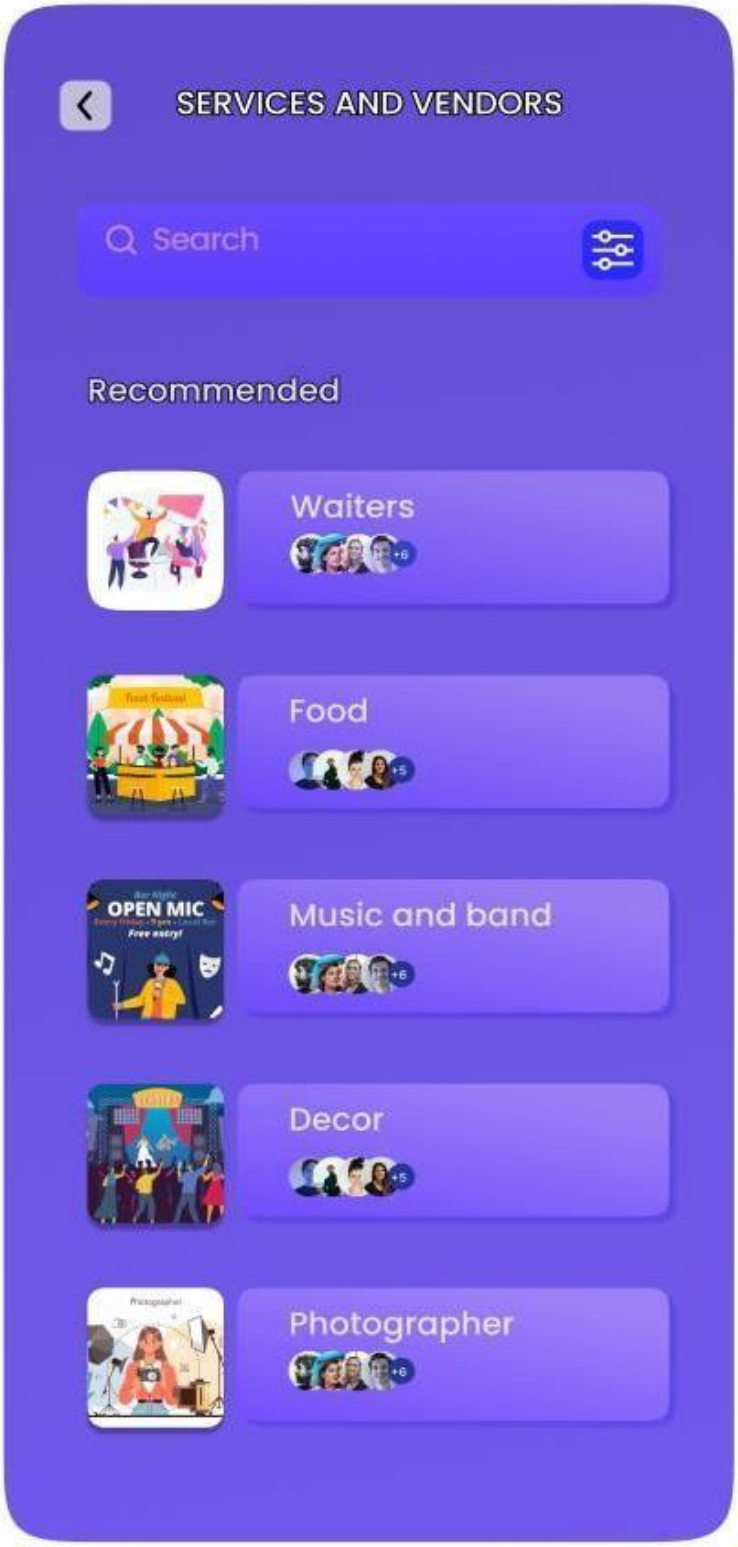


Figure 2-22 Services and Vendors

Figure 2-23 Vendor Profile

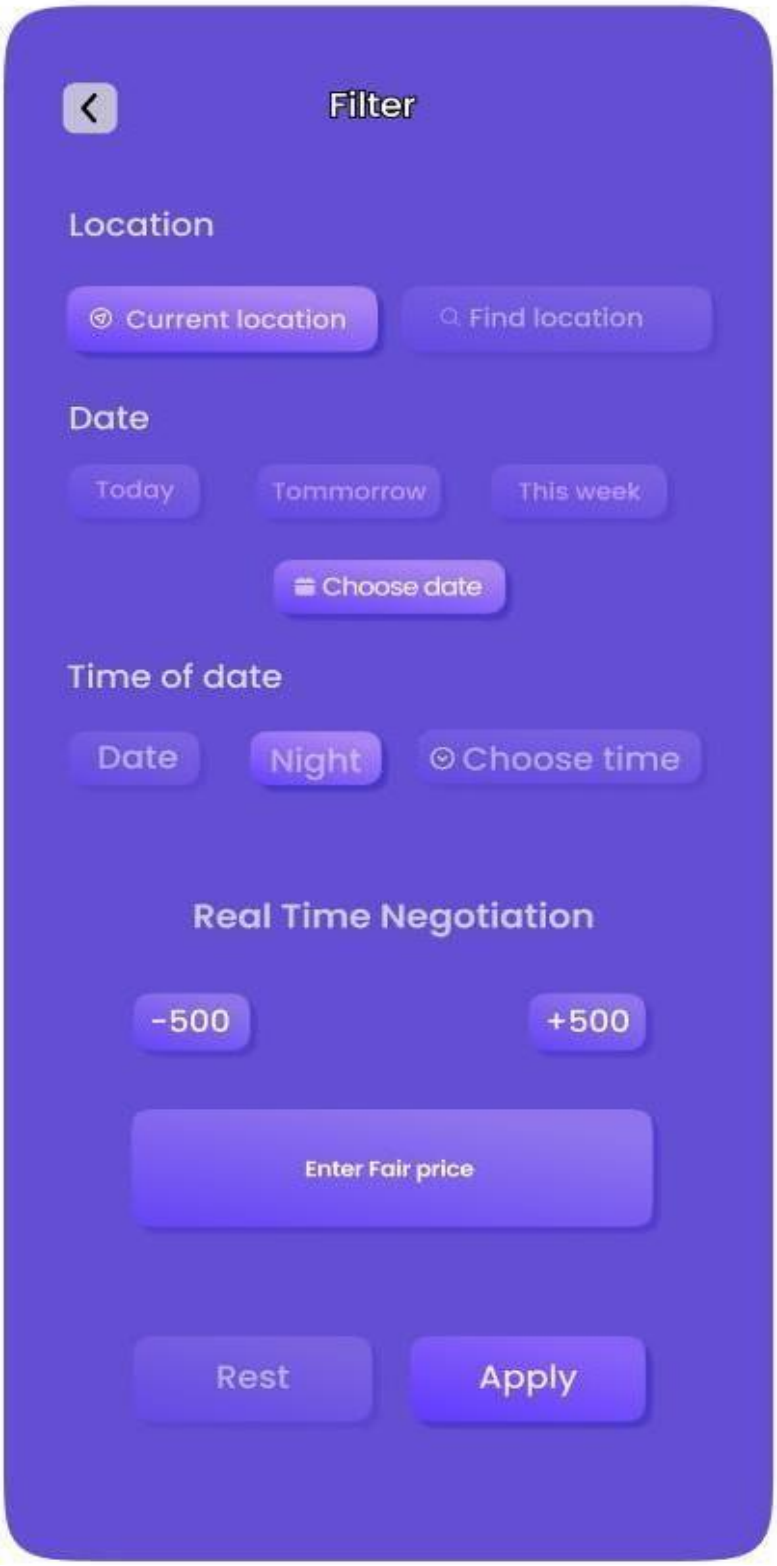


Figure 2-24 Filter

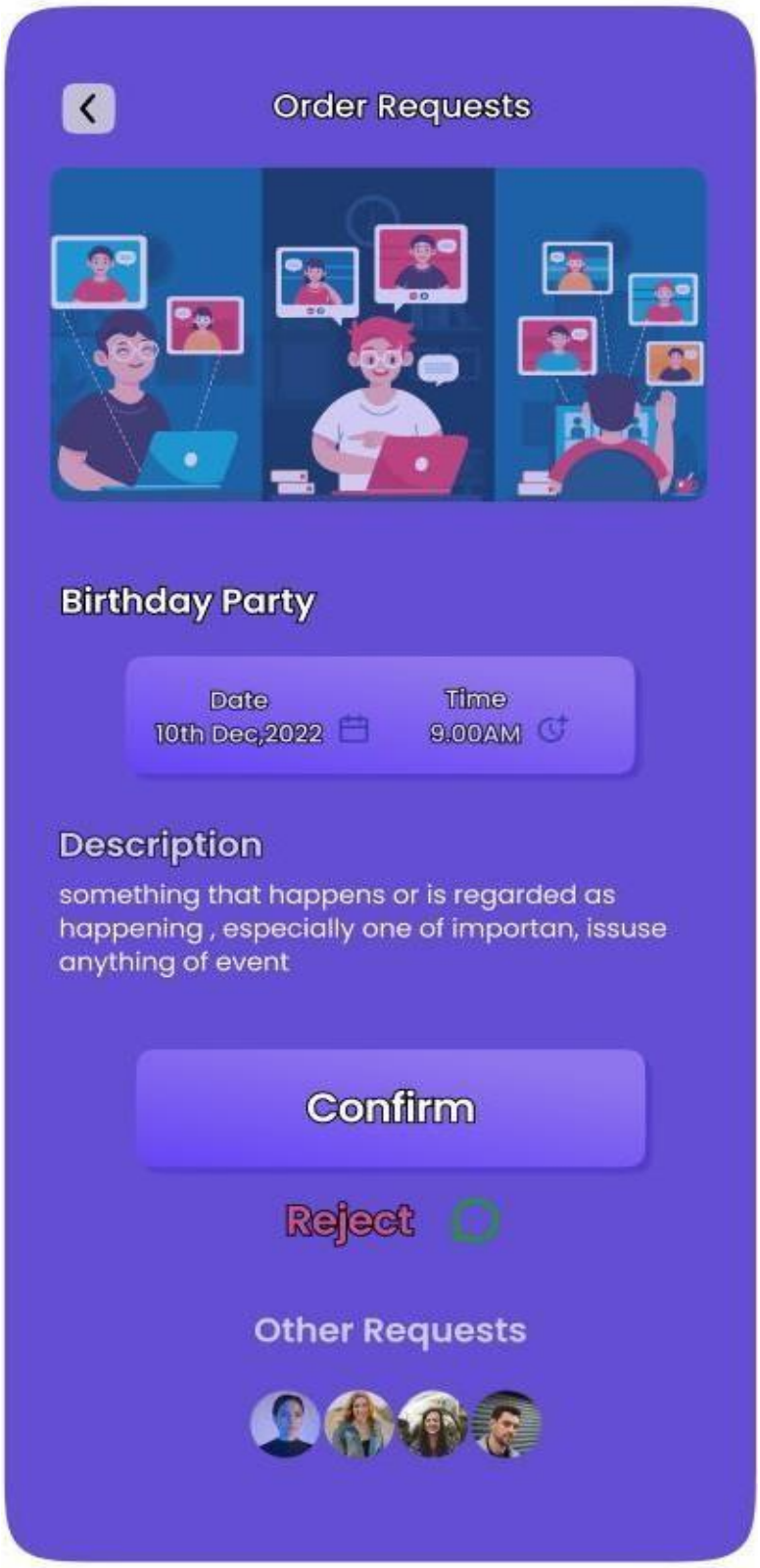


Figure 2-25 Order Request

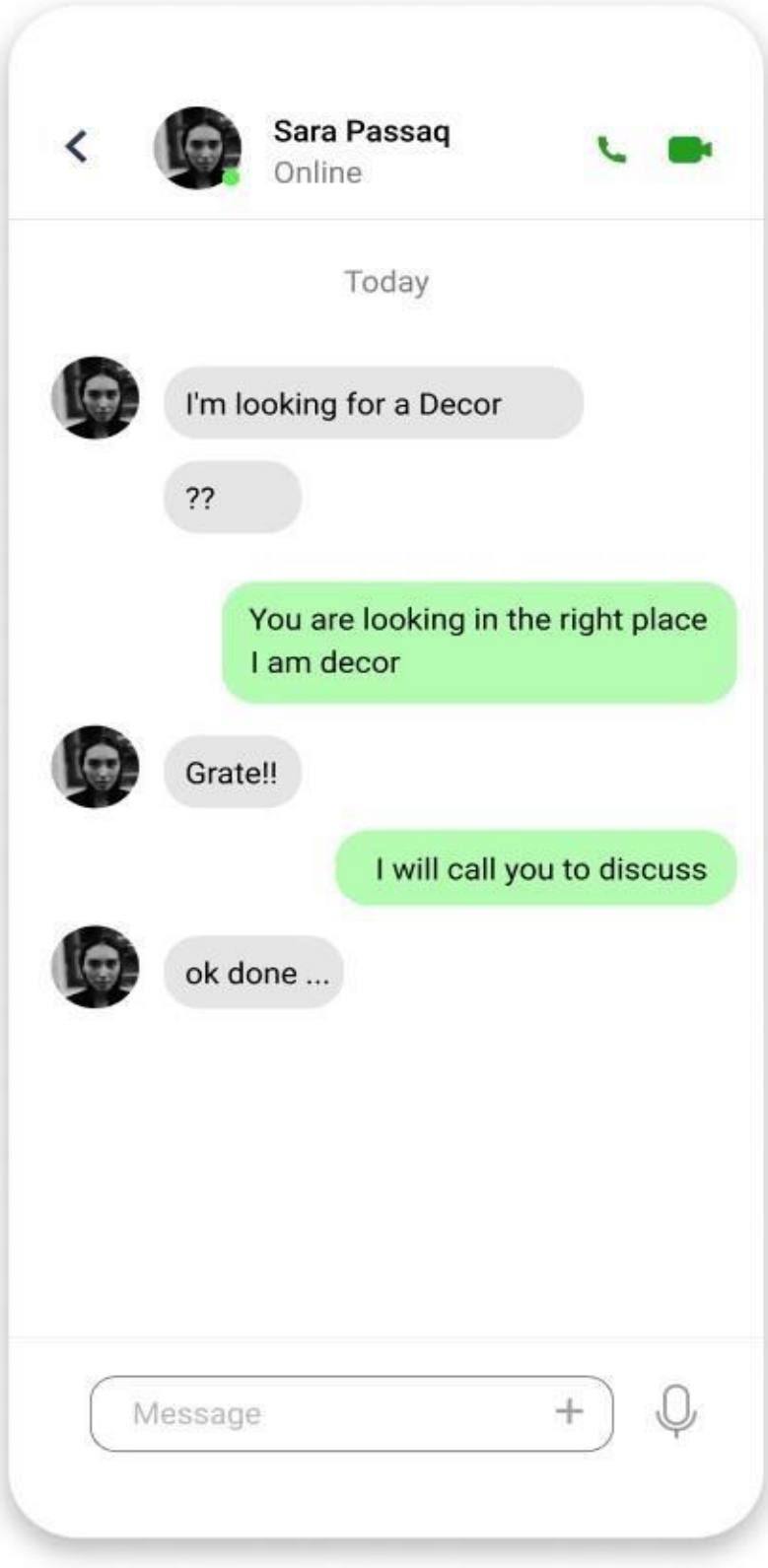


Figure 2-26 User and Vendor Chat

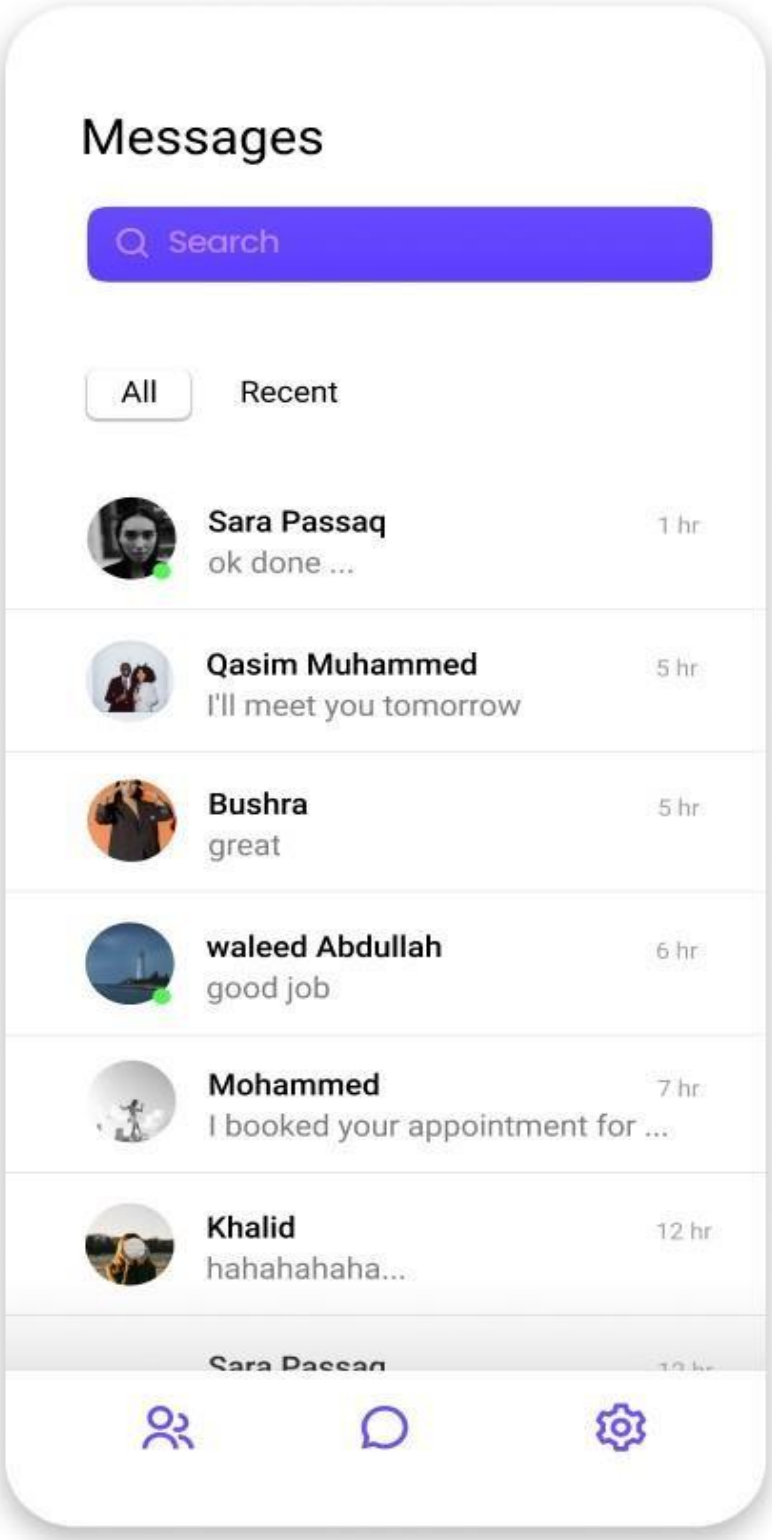


Figure 2-27 Message Chat list

Chapter 3

3. System Design

3.1. Software Architecture

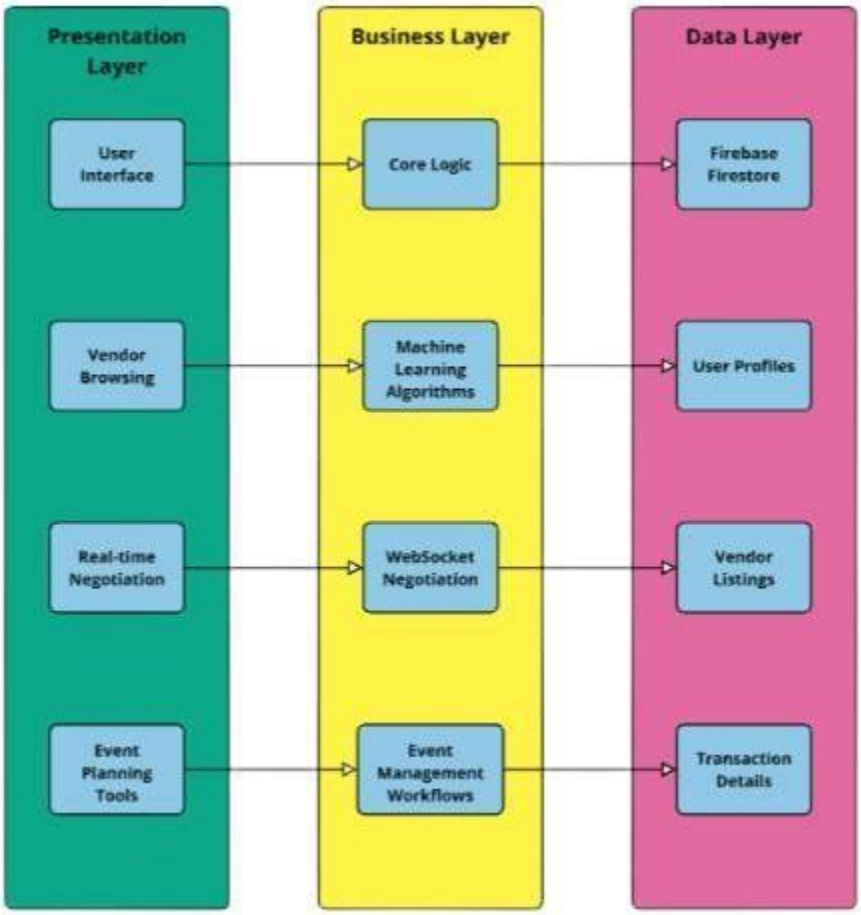


Figure 3-1 Software Architecture

3.2. Class Diagram

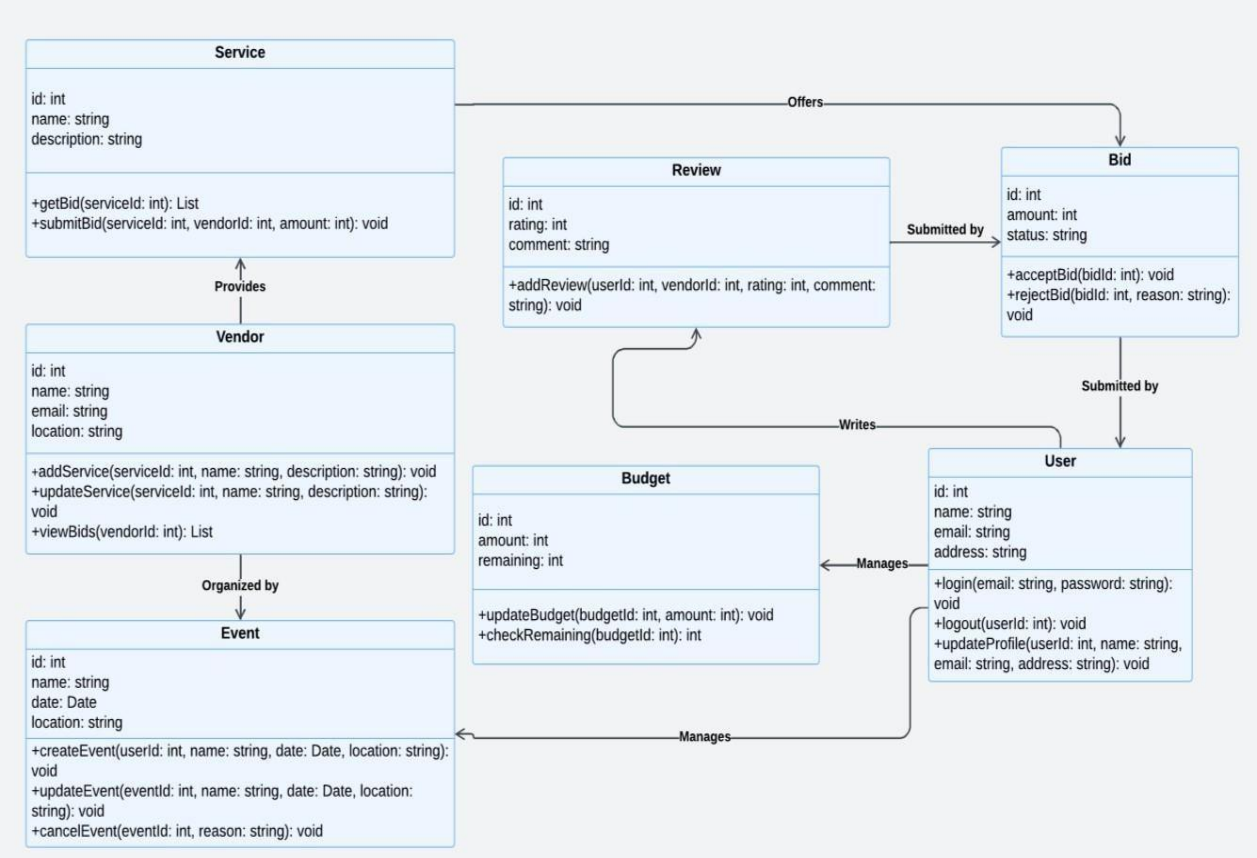


Figure 3-2 Class Diagram

3.3. Sequence Diagram

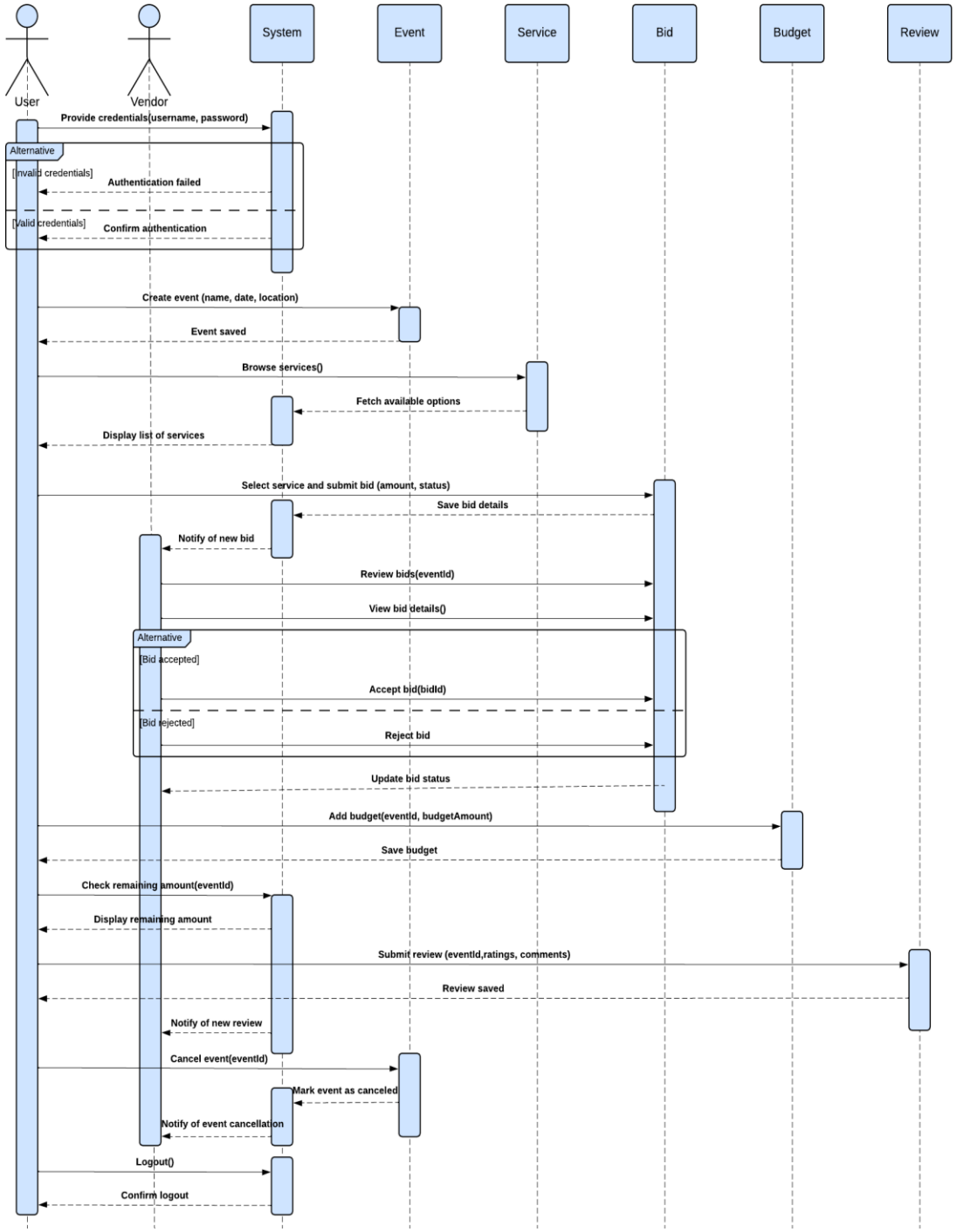


Figure 3-3 Sequence Diagram

3.4. Entity Relationship Diagram

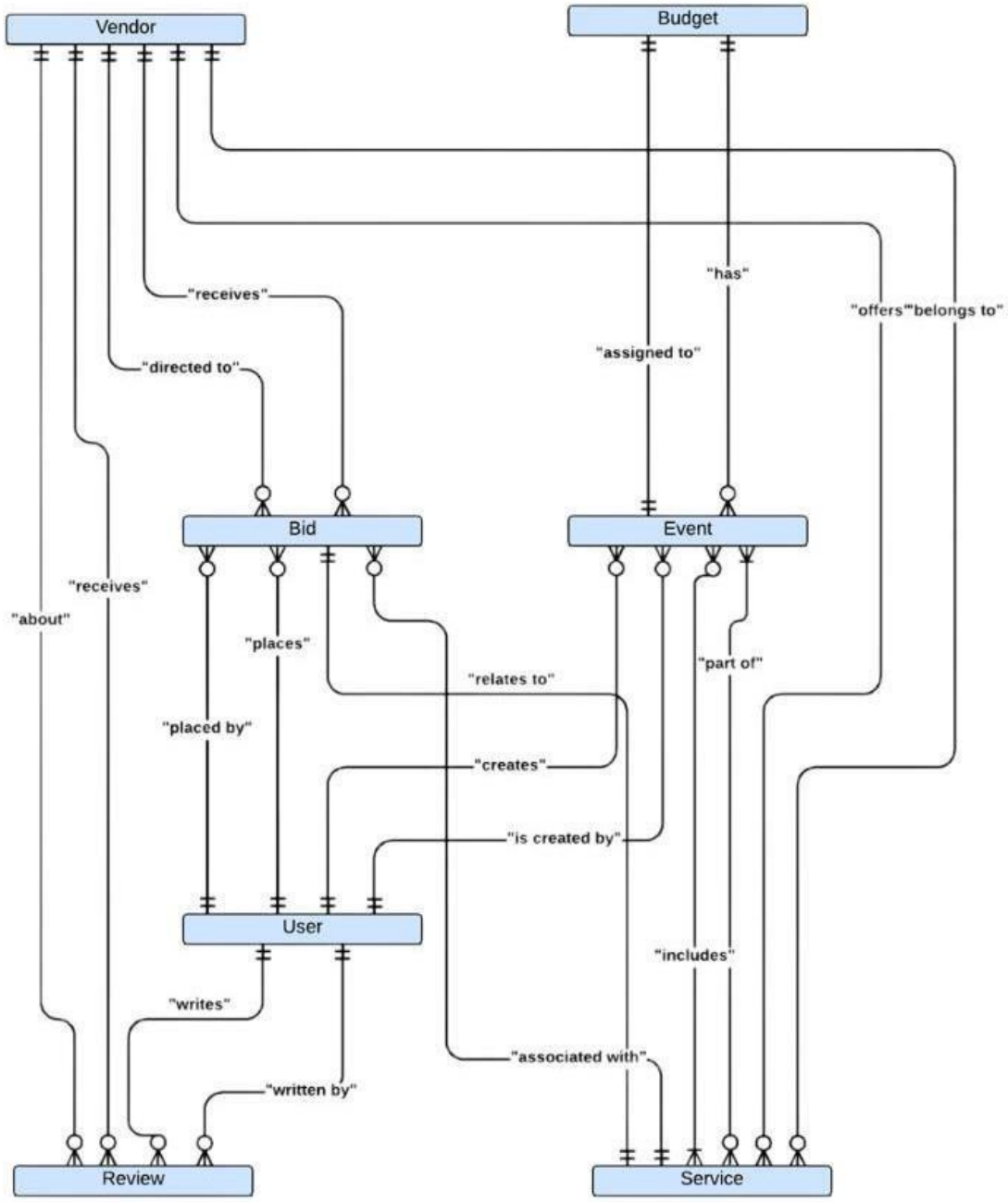


Figure 3-4 Entity Relationship Diagram

3.5. Database Schema

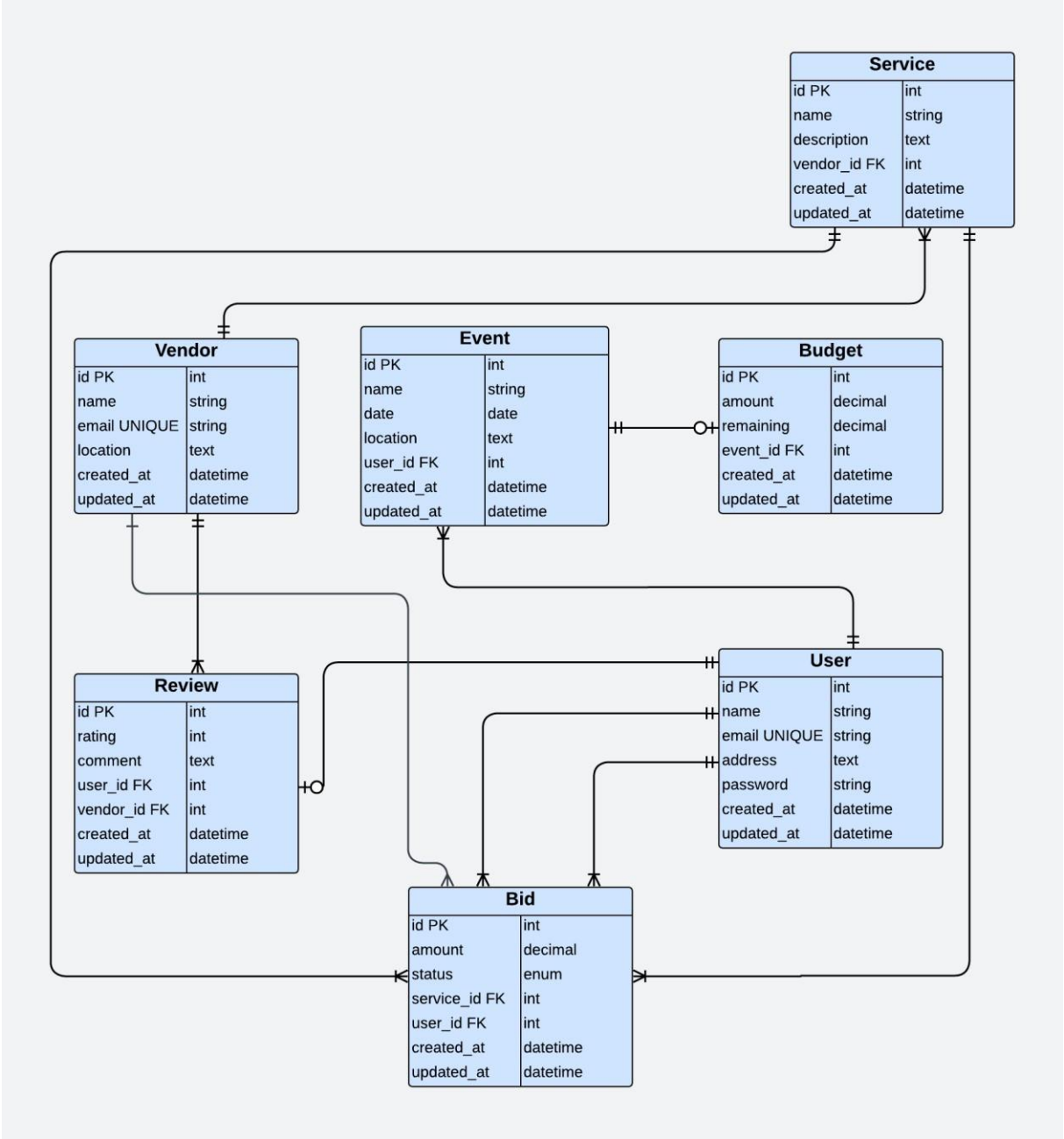


Figure 3-5 Database Schema

Chapter 4

4. Software Development

4.1. Coding Standards

Indentation:
The project follows a standard indentation of four spaces per level, ensuring readability and consistent formatting. Nested structures like classes, methods, and conditionals are indented appropriately.

Declaration:
Variables are declared with clear and descriptive names, following camelCase notation (e.g., `eventNameEditText`, `vendorLoginButton`). Class names use PascalCase (e.g., `VendorLogin`, `Home`).

Naming Convention:

- **Classes:**
  PascalCase (e.g. DBHelper, VendorLogin).

- **Variables:**
  eventLocationEditText (representing an edit text for event location) follows camelCase as well.

- **Constants:**
  MY*PERMISSIONS*REQUEST_LOCATION is a constant which follows Uppercase with Underscores style.

- **Methods:**
  showDatePicker and validateUser are examples of methods which also follow camelCase.

Statement Standards:

- Statements are written on separate lines, each ending with a semicolon.
- Control structures such as `if-else` and loops use proper braces `{}` even for single-line blocks, promoting clarity and preventing errors.

4.2. Development Environment

Tools and Technologies:

- **IDE:**

  Android Studio, chosen for its robust tools for Android development, integrated Gradle build system, and debugging capabilities.

- **Language:**

  Java, selected for its extensive support for Android development and familiarity among developers.

- **Database:**

  SQLite, integrated via custom helpers like DBHelper and VendorDBHelper, for local data persistence.

Development Environment Setup:
The environment was set up by installing Android Studio, configuring the SDK, and setting up an emulator for testing. Physical devices were also used for testing the app in real-world scenarios.

Packages Used:

- **Google Maps API:**

  For location-based features like vendor search and map integration.

- **AndroidX Libraries:**

  For modern UI components and backward compatibility.

- **SharedPreferences:**

  For lightweight local storage of user and event data.

Third-Party Libraries:

- **Geocoder:**

  Used for converting addresses into geographical coordinates.

- **Google Maps:**

  Integrated for location visualization.

4.3. Software Description

4.3.1. Snippet 1 (Customer Login)

```
package com.example.anew;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.example.New.DBHelper;
public class login extends AppCompatActivity {
    private EditText username, password;
    private Button loginButton;
    private TextView signupRedirect;
    private DBHelper dbHelper;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);
        // Initialize views
        username = findViewById(R.id.text_1);
        password = findViewById(R.id.text_2);
        loginButton = findViewById(R.id.loginButton);
        signupRedirect = findViewById(R.id.sign_1);

        // Initialize DBHelper
        dbHelper = new DBHelper(this);
        // Handle login button click
        loginButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
```

```
                // Get user input
                String enteredUsername =
username.getText().toString().trim();
                String enteredPassword =
password.getText().toString().trim();
                // Validate input
                if (enteredUsername.isEmpty() ||
enteredPassword.isEmpty()) {
                    Toast.makeText(login.this, "Please fill in all
fields", Toast.LENGTH_SHORT).show();
                } else {
                    // Check credentials in the database
                    if (dbHelper.validateUser(enteredUsername,
enteredPassword)) {
                        Toast.makeText(login.this, "Login
Successful!", Toast.LENGTH_SHORT).show();
                        // Redirect to the home activity
        Intent intent = new Intent(login.this, home.class);
                        startActivity(intent);
                        finish();
                    } else {
Toast.makeText(login.this, "Invalid Username or Password",
Toast.LENGTH_SHORT).show();
                    } } } });
        // Redirect to the signup page
        signupRedirect.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(login.this, signup.class);
                startActivity(intent);
            }
        });
    }
}
```

Description:

The following code implements user login functionality. It checks username and password against a database (DBHelper) to confirm if the user exists. If successful, the user is taken to the home screen. If not successful, an error message is shown. There is also an option for new users to go to the Signup page.

4.3.2. Snippet 2 (Vendor Login)

```
package com.example.anew;

import android.annotation.SuppressLint;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.example.New.VendorDBHelper;

public class vendorlogin extends AppCompatActivity {

    private EditText username, password;
    private Button loginButton;
    private TextView signupRedirect;
    private VendorDBHelper dbHelper;

    @SuppressLint("MissingInflatedId")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_vendorlogin);

        username = findViewById(R.id.vendorloginusername);
        password = findViewById(R.id.vendorloginpassword);
        loginButton = findViewById(R.id.vendorloginbutton);
        signupRedirect = findViewById(R.id.sign_2);

        dbHelper = new VendorDBHelper(this);

       loginButton.setOnClickListener(new View.OnClickListener() {
            @Override
         public void onClick(View v) {
                String enteredUsername =
username.getText().toString().trim();
                String enteredPassword =
password.getText().toString().trim();

                if (enteredUsername.isEmpty() ||
enteredPassword.isEmpty()) {
                    Toast.makeText(vendorlogin.this, "Please fill in
```

```
all fields", Toast.LENGTH_SHORT).show();
                } else {
                    if (dbHelper.validateVendor(enteredUsername,
enteredPassword)) {
                        Toast.makeText(vendorlogin.this, "Login
Successful!", Toast.LENGTH_SHORT).show();
                        Intent intent = new Intent(vendorlogin.this,
vendorhome.class);
                        startActivity(intent);

                    } else {
                        Toast.makeText(vendorlogin.this, "Invalid
Username or Password", Toast.LENGTH_SHORT).show();
                    }
                } }
        });

        signupRedirect.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(vendorlogin.this,
vendorsignup.class);
                startActivity(intent);
            }
        });
    }
}
```

Description:

This code is responsible for the login of the vendor. He checks the username and password fields against the database using a VendorDBHelper and if they are correct, he forwards the vendor to the vendorhome screen. In case of error he returns an error message. For new vendors, there is a link on the signup page so they can easily register.

4.3.3. Snippet 3 (Home Page)

```
package com.example.anew;

        view.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(home.this,
ViewEventActivity.class);
                startActivity(intent);
            }
        });
        schedule.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(home.this,
scheduleEvent.class);
                startActivity(intent);
            }
        });
        viewschedule.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(home.this,
Viewschedule.class);
                startActivity(intent);
            }
        });
        findtheoffer.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(home.this, Bidding.class);
                startActivity(intent);

            }
        });
    }
            }
        });
```

Description:

This code defines the home activity, which serves as the main screen for navigation. It provides options for users to:

1. **Plan an Event**:
   Redirects to the addEvent activity to create a new event.
2. **View Events**:
   Opens the ViewEventActivity to display existing events.

3. **Schedule an Event**:
   Navigates to the scheduleEvent activity for scheduling an event.
4. **View Scheduled Events**:
   Opens the Viewschedule activity to view scheduled events.
5. **Find Offers**:
   Redirects to the Bidding activity for finding and interacting with offers.

4.3.4. Snippet 4 (Schedule Event)

```
package com.example.anew;

import android.annotation.SuppressLint;
import android.app.DatePickerDialog;
import android.content.Intent;
import android.content.SharedPreferences;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import java.util.Calendar;

public class scheduleEvent extends AppCompatActivity {

    private EditText eventNameEditText, eventDateEditText,
eventTimeEditText, eventLocationEditText, eventDescriptionEditText;
    private ImageButton calendarButton;

    @SuppressLint("MissingInflatedId")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_schedule_event);

        // Initialize views
        eventNameEditText = findViewById(R.id.event_name);
        eventDateEditText = findViewById(R.id.date);
        calendarButton = findViewById(R.id.calendarButton);
        eventTimeEditText = findViewById(R.id.event_time);
        eventLocationEditText = findViewById(R.id.event_location);
        eventDescriptionEditText =
findViewById(R.id.event_description);
        Button saveEventButton = findViewById(R.id.save_event_button);

        // Disable editing of the date EditText
        eventDateEditText.setFocusable(false);

        // Handle calendar button click
        calendarButton.setOnClickListener(new View.OnClickListener() {
```

```
            @Override
            public void onClick(View v) {
                showDatePicker();
            }
        });

        // Handle date EditText click
        eventDateEditText.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                showDatePicker();
            }
        });

        // Set click listener for the save button
        saveEventButton.setOnClickListener(new View.OnClickListener()
{
            @Override
            public void onClick(View v) {
                String eventName =
eventNameEditText.getText().toString().trim();
                String eventDate =
eventDateEditText.getText().toString().trim();
                String eventTime =
eventTimeEditText.getText().toString().trim();
                String eventLocation =
eventLocationEditText.getText().toString().trim();
                String eventDescription =
eventDescriptionEditText.getText().toString().trim();

                // Validate input
                if (TextUtils.isEmpty(eventName) ||
TextUtils.isEmpty(eventDate) ||
                        TextUtils.isEmpty(eventTime) ||
TextUtils.isEmpty(eventLocation)) {
                    Toast.makeText(scheduleEvent.this, "Please fill in
all required fields", Toast.LENGTH_SHORT).show();
                } else {
                    // Save event details to SharedPreferences
                    saveEventToPreferences(eventName, eventDate,
eventTime, eventLocation, eventDescription);

                    // Navigate to ViewScheduleActivity
                    Intent intent = new Intent(scheduleEvent.this,
Viewschedule.class);
                    startActivity(intent);

                    // Close the current activity
                    finish();
                }
            }
        });
    }

    // Method to save event details to SharedPreferences
```

```
    private void saveEventToPreferences(String name, String date,
String time, String location, String description) {
        SharedPreferences sharedPreferences =
getSharedPreferences("EventSchedulePrefs", MODE_PRIVATE);
        SharedPreferences.Editor editor = sharedPreferences.edit();
        editor.putString("event_name", name);
        editor.putString("event_date", date);
        editor.putString("event_time", time);
        editor.putString("event_location", location);
        editor.putString("event_description", description);
        editor.apply();
    }

    // Method to show DatePickerDialog
    private void showDatePicker() {
        // Get the current date
        final Calendar calendar = Calendar.getInstance();
        int year = calendar.get(Calendar.YEAR);
        int month = calendar.get(Calendar.MONTH);
        int day = calendar.get(Calendar.DAY_OF_MONTH);

        // Create a DatePickerDialog
        DatePickerDialog datePickerDialog = new DatePickerDialog(this,
(view, selectedYear, selectedMonth, selectedDay) -> {
            // Set selected date in EditText
            String formattedDate = selectedYear + "-" +
String.format("%02d", (selectedMonth + 1)) + "-" +
String.format("%02d", selectedDay);
            eventDateEditText.setText(formattedDate);
        }, year, month, day);

        datePickerDialog.show();
    }
}
```

Description:

This code implements the functionality for scheduling an event in an Android app. Here's a brief description:

1. **Input Fields**:

   Users can input event details like name, date, time, location, and description.

2. **Date Picker**:

   The date field is non-editable, and a date picker dialog is used to select a date conveniently.

3. **Save Event**:

    - When the save button is clicked, it validates that all required fields are filled.
    - If valid, the event details are saved to SharedPreferences for persistence.

4. **Navigation**:

    After saving the event, the user is redirected to the Viewschedule activity to view the scheduled events.

4.3.5 Snippet 5 (Bidding)

```
package com.example.anew;

import android.Manifest;
import android.annotation.SuppressLint;
import android.content.pm.PackageManager;
import android.location.Address;
import android.location.Geocoder;
import android.location.Location;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

import com.example.anew.R;
import com.google.android.gms.maps.CameraUpdate;
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.MarkerOptions;

import java.io.IOException;
import java.util.List;
import java.util.Locale;

public class Bidding extends AppCompatActivity implements
OnMapReadyCallback {
    static final int MY_PERMISSIONS_REQUEST_LOCATION = 23;
    private GoogleMap mMap;

    private EditText findAddress;
```

```
    private TextView yourOffer, offerAmount;
    private Button btnMinus, btnPlus, btnFindVendor, btnSearchAddress;
    private ImageView imageView;

    private int offerAmountValue = 50000; // Initial offer amount

    @SuppressLint("MissingInflatedId")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_bidding);

        // Initialize views
        findAddress = findViewById(R.id.address);
        yourOffer = findViewById(R.id.yourOffer);
        offerAmount = findViewById(R.id.offerAmount);
        btnMinus = findViewById(R.id.Minus);
        btnPlus = findViewById(R.id.plus);
        btnFindVendor = findViewById(R.id.findvendor);
        btnSearchAddress = findViewById(R.id.butt);

        // Set initial offer amount
        updateOfferAmount();
        checkPermission();

        // Button click listeners for offer adjustments
        btnMinus.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                decreaseOfferAmount();
            }
        });

        btnPlus.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                increaseOfferAmount();
            }
        });

        btnFindVendor.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                findVendor();
            }
        });

        // Search address and display it on the map
        btnSearchAddress.setOnClickListener(new View.OnClickListener()
{
            @Override
            public void onClick(View v) {
                String address =
findAddress.getText().toString().trim();
                if (!address.isEmpty()) {
                    findAddressOnMap(address);
```

```
                } else {
                    Toast.makeText(Bidding.this, "Please enter an
address!", Toast.LENGTH_SHORT).show();
                }
            }
        });
    }
  private void checkPermission() {
        if (ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_FINE_LOCATION)
                != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this, new
String[]{Manifest.permission.ACCESS_FINE_LOCATION},
                    MY_PERMISSIONS_REQUEST_LOCATION);
        } else {
            SupportMapFragment mapFragment = (SupportMapFragment)
getSupportFragmentManager().findFragmentById(R.id.map);
            if (mapFragment != null) {
                mapFragment.getMapAsync(this);
            }
        }
    }
    @SuppressLint("MissingSuperCall")
    @Override
    public void onRequestPermissionsResult(int requestCode, @NonNull
String[] permissions, @NonNull int[] grantResults) {
        if (requestCode == MY_PERMISSIONS_REQUEST_LOCATION) {
            if (grantResults.length > 0 && grantResults[0] ==
PackageManager.PERMISSION_GRANTED) {
                SupportMapFragment mapFragment = (SupportMapFragment)
getSupportFragmentManager().findFragmentById(R.id.map);
                if (mapFragment != null) {
                    mapFragment.getMapAsync(this);
                }
            } else {
                Toast.makeText(this, "Location permission denied",
Toast.LENGTH_SHORT).show();
            }
        }
    }
    @Override
    public void onMapReady(GoogleMap googleMap) {
        mMap = googleMap;

        // Set an initial location
        LatLng initialLocation = new LatLng(33.6844, 73.0479); //
Islamabad

mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(initialLocation,
12));
    }

    private void findAddressOnMap(String address) {
        if (mMap == null) {
            Toast.makeText(this, "Map is not ready yet!",
Toast.LENGTH_SHORT).show();
```

```
            return;
        }

        Geocoder geocoder = new Geocoder(this, Locale.getDefault());
        try {
            List<Address> addressList =
geocoder.getFromLocationName(address, 1);
            if (addressList != null && !addressList.isEmpty()) {
                Address location = addressList.get(0);
                LatLng latLng = new LatLng(location.getLatitude(),
location.getLongitude());

                mMap.clear(); // Clear existing markers
                mMap.addMarker(new
MarkerOptions().position(latLng).title(address));

mMap.animateCamera(CameraUpdateFactory.newLatLngZoom(latLng, 15));
                Toast.makeText(this, "Location found: " + address,
Toast.LENGTH_SHORT).show();
            } else {
                Toast.makeText(this, "Address not found!",
Toast.LENGTH_SHORT).show();
            }
        } catch (IOException e) {
            Toast.makeText(this, "Error finding address: " +
e.getMessage(), Toast.LENGTH_SHORT).show();
        }
    }

    private void updateOfferAmount() {
        offerAmount.setText(String.format("PKR %d",
offerAmountValue));
    }

    private void decreaseOfferAmount() {
        if (offerAmountValue > 2000) {
            offerAmountValue -= 1000; // Decrease by 1000
            updateOfferAmount();
        } else {
            Toast.makeText(this, "Minimum offer reached!",
Toast.LENGTH_SHORT).show();
        }
    }

    private void increaseOfferAmount() {
        offerAmountValue += 1000; // Increase by 1000
        updateOfferAmount();
    }

    private void findVendor() {
        Toast.makeText(this, "Finding the nearest vendor for PKR " +
offerAmountValue, Toast.LENGTH_SHORT).show();
        // Add logic to find vendor (e.g., network call or other
action)
    }
}
```

Description:

This code defines the functionality for a bidding system in an Android app. Here's a brief explanation:

1. **Offer Management**:

   Users can adjust the initial offer amount (PKR 50,000) using the "+" and "-" buttons.

2. **Find Vendor**:

   A button allows users to initiate a search for vendors based on the current offer amount.

3. **Address Search**:

   - Users can input an address to locate it on the integrated Google Map.
   - The address is geocoded into coordinates, and a marker is displayed on the map.

4. **Google Maps Integration**:

   - The app includes a map to display locations.
   - The app requests location permissions, and an initial default location (Islamabad) is shown.

5. **Error Handling**:

   Displays appropriate messages if permissions are denied, the map is unavailable, or the address search fails.

4.3.6 Snippet 6 (Vendor Home)

```
package com.example.anew;

import android.annotation.SuppressLint;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import com.example.anew.Bidding;
import com.example.anew.Bidding;

public class vendorhome extends AppCompatActivity {
    @SuppressLint("MissingInflatedId")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_vendorhome);
        View vendorview=findViewById(R.id.vendorviewevent);
        View vendorviewschedule= findViewById(R.id.vendorviewschedule);

       vendorview.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(vendorhome.this,
vendorviewevent.class);
                startActivity(intent);
            }});

        vendorviewschedule.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(vendorhome.this,
vendorviewschedule.class);
                startActivity(intent);} }); }}
```

Description:

This code defines the vendorhome activity, which serves as the main screen for vendors. Here's a brief description:

1. **Navigation Options**:
    - **View Events**:
      Redirects to the vendorviewevent activity, allowing vendors to view the events they are involved in.

- **View Schedule**:
  Navigates to the vendorviewschedule activity to display the vendor's schedule.

2. **Edge-to-Edge Design**:

   The EdgeToEdge.enable() method is used to enable edge-to-edge screen layouts for a modern, immersive user experience.

Chapter 5

5. Software Testing

Software testing is an important and necessary step in making software. It acts as the final check to make sure the system not only meets the basic needs but also works well and has no serious problems. For the BudgetBliss Events platform, careful testing was needed to make sure all parts work correctly and the system is easy to use before it is given to users.

The main goal of testing was to check if each feature and part, as well as the whole system together, worked as expected. This helped find and fix any problems, making the system stronger and better for users.

5.1. Testing Methodology

To achieve a thorough evaluation of BudgetBliss, we adopted a multi-layered testing approach. Each type of testing served a distinct purpose and was applied at appropriate stages during the development to catch issues early and systematically improve the software’s quality:

- **Unit Testing:**

  This was the foundational step where individual components were tested in isolation. For example, the login and signup modules were verified to ensure users could authenticate correctly; event scheduling was checked for accuracy in setting dates and times; and the bidding module was tested to confirm that users could

place bids without errors. By focusing on single units, any faults could be quickly identified and fixed before they affected other parts of the system.

- **Integration Testing:**

  The next step I took after making sure that each unit worked correctly on its own was to test the integration of the individual units.This was crucial in a complex system like BudgetBliss, where modules such as vendor profile management had to seamlessly connect with event scheduling and bidding functionalities. Integration testing helped uncover interface issues or data inconsistencies that could arise when modules communicate, ensuring that the system components worked together as intended.

- **System Testing:**

  At this stage, the platform was evaluated as a whole entity. System testing involved simulating real-world user scenarios—from registering new users and creating events to sharing documents and negotiating bids—making sure that every feature collectively met the defined requirements. This end-to-end validation was essential to confirm the overall stability, usability, and performance of the system.

- **Real-Time Testing:**

  One of BudgetBliss's key features is real-time price negotiation via WebSockets, enabling instantaneous communication between users and vendors. Testing this real-time functionality was critical to ensure that messages, bids, and counteroffers were transmitted quickly and reliably without delays or data loss. This testing helped guarantee a fluid and responsive user experience during negotiations.

- **Map and Location Testing:**

  The integration of Google Maps to visualize event locations was another important feature. We verified that addresses entered by users were accurately converted into geographical coordinates using the Geocoder library and correctly displayed on the map interface. This testing ensured that users could trust the location information provided and navigate easily.

By using these different testing methods, we were able to find and fix problems early during development. This helped make BudgetBliss a more reliable and higher-quality system.

5.2. Testing Environment

To simulate conditions as close as possible to real-world usage, the testing environment was carefully chosen and set up to replicate the experience of actual end-users:

- **Development Tools:**

  Testing was primarily conducted using Android Studio, which provided a robust platform for both development and debugging.

- **Operating System:**

  The tests were performed on a Windows machine, mirroring the common environment used during development.

- **Devices:**

  Testing utilized both Android emulators and physical Android devices, including popular models such as Samsung and Vivo smartphones. This combination allowed us to check how the application performed across different screen sizes, hardware capabilities, and OS versions, ensuring compatibility and responsiveness.

- **Backend Infrastructure:**

  Firebase Firestore was used as the real-time database, allowing us to test data synchronization and retrieval in a way that closely mirrors live conditions. Hosting on Google Cloud ensured the backend infrastructure was stable and scalable.

- **Libraries and APIs:**

  - The **Google Maps API** was utilized to display event locations visually on the

map.

- The **Geocoder library** helped in converting user-input addresses into latitude and longitude coordinates for accurate mapping.
- **SharedPreferences** were tested for their role in storing user preferences and local data on the device, verifying data persistence between sessions.

This comprehensive testing setup enabled us to carefully monitor the application's performance, identify potential issues, and fine-tune the software for a seamless user experience under a variety of real-life conditions.

5.3. Test Cases

5.3.1. Test Case 1 (User Registration and Authentication)

Table 5- 1 User Registration and Authentication

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Ensure users can securely register and log in	*Test ID:* TC-01
Version: 1	*Test Type:* Unit Testing
Input: Username:testuser Password: test123	
Expected Result: User account is created and authenticated	
Actual Result: **Passed**	

5.3.2. Test Case 2 (Invalid Login)

Table 5-2 Invalid Login (Incorrect Credentials)

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Ensure login fails with incorrect credentials	***Test ID:***TC-2
Version:1	***Test Type:*** Functional Testing
Input: **Username:**wronguser **Password:** wrongpass	
Expected Result: Error message: 'Invalid Username or Password'	
Actual Result: **Failed**	

5.3.3. Test Case 3 (Vendor Management Module)

Table 5- 3 Vendor Management Module

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Vendors can register, create/update profile	***Test ID:***TC-3
Version:1	***Test Type:*** Functional Testing
Input: Vendor info: name, email, service details	
Expected Result: Vendor profile updated/created and visible to users	
Actual Result: **Passed**	

5.3.4. Test Case 4 (Vendor Profile Submission)

Table 5- 4 Vendor Profile Submission with Missing Information

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Vendor submits incomplete profile	***Test ID:***TC-4
Version:1	***Test Type:*** UI Testing
Input: Missing service description	
Expected Result: System should prevent submission and prompt for missing fields	
Actual Result: **Failed**	

5.3.5. Test Case 5 (User Profile Managaement)

Table 5- 5 User Profile Management

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Allow users to edit profile and preferences	***Test ID:***TC-5
Version:1	***Test Type:*** Unit Testing
Input: New profile picture and updated bio	
Expected Result: Changes are reflected in the user profile	
Actual Result: **Passed**	

5.3.6. Test Case 6 (Profile Picture Upload)

Table 5- 6 Profile Picture Upload with Unsupported File Format

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Upload unsupported file type for profile picture	***Test ID:***TC-7
Version:1	***Test Type:*** Validation Testing
Input: Upload `.exe` file	
Expected Result: System should reject the upload with an error	
Actual Result: **Failed**	

5.3.7. Test Case 7 (Vendor Availability Calendar)

Table 5- 7 Vendor Availability Calendar

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Vendor marks available/unavailable dates	***Test ID:***TC-7
Version:1	***Test Type:*** Unit Testing
Input: **Mark:** Jan 5–10 available	
Expected Result: Calendar updated & visible to users	
Actual Result: **Passed**	

5.3.8. Test Case 8 (Search and Filtering System)

Table 5- 8 Search and Filtering System

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Filter vendors by budget, category, location	***Test ID:***TC-08
Version:1	***Test Type:*** Functional Testing
Input: **Budget:** < PKR 50,000, **Category:** Catering	
Expected Result: Filtered list is shown	
Actual Result: **Passed**	

5.3.9. Test Case 9 (Real-Time Price Negotiation)

Table 5-9 Real-Time Price Negotiation

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Enable price negotiation between user and vendor	***Test ID:***TC-9
Version:1	***Test Type:*** Integration Testing
Input: **Offer:** PKR 42,000 (from base 50,000)	
Expected Result: Vendor responds; negotiation completes	
Actual Result: **Passed**	

5.3.10. Test Case 10 (Personalized Event Recommendations)

Table 5-10 P Vendor Filtering by Budget and Category

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Filter vendors manually by budget range and service category	***Test ID:***TC-10
Version:1	***Test Type:*** Functional Testing
Input: **User:** birthday event, PKR 40,000	
Expected Result: List of photographers available in Islamabad with packages under or equal to PKR 40,000 is displayed.	
Actual Result: **Passed**	

5.3.11. Test Case 11 (Budget Management Tools)

Table 5-11 Budget Management Tools

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Monitor and adjust event budget	***Test ID:***TC-11
Version:1	***Test Type:*** Functional Testing
Input: **Add:** Catering PKR 30,000 to PKR 100,000	
Expected Result: Budget adjusts to PKR 70,000 remaining	
Actual Result: **Passed**	

5.3.12. Test Case 12 (Vendor Ratings and Review System)

Table 5-12 Vendor Ratings and Review System

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Users post ratings after events	***Test ID:***TC-12
Version:1	***Test Type:*** Unit Testing
Input: 5-star rating and review	
Expected Result: Review saved and visible	
Actual Result: **Passed**	

5.3.13. Test Case 13 (Event Planning Dashboard)

Table 5-13 Event Planning Dashboard

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Users track event progress from dashboard	***Test ID:***TC-13
Version:1	***Test Type:*** UI Testing
Input: Create new event & view progress	
Expected Result: Tasks, expenses visible	
Actual Result: **Passed**	

5.3.14. Test Case 14 (Communication Interface)

Table 5-14 Communication Interface

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Real-time messaging between users and vendors	***Test ID:***TC-14
Version:1	***Test Type:*** Integration Testing
Input: Is your service available on Jan 1st?	
Expected Result: Vendor receives and replies instantly	
Actual Result: **Passed**	

5.3.15. Test Case 15 (Event Scheduling and Notifications)

Table 5-15 Event Scheduling and Notifications

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Schedule tasks with reminders	***Test ID:***TC-15
Version:1	***Test Type:*** Functional Testing
Input: **Task:** "Book Decorator"**, Reminder:** 1 day before	
Expected Result: Task added and notification appears	
Actual Result: **Passed**	

5.3.16. Test Case 16 (Service Booking Confirmation)

Table 5-16 Service Booking Confirmation

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Confirm vendor bookings after negotiation	***Test ID:*** TC-16
Version: 1	***Test Type:*** UI Testing
Input: **Confirm Booking:** Catering PKR 45,000	
Expected Result: Confirmation sent to user and vendor	
Actual Result: **Passed**	

5.3.17. Test Case 17 (Document Upload and Sharing)

Table 5-17 Document Upload and Sharing

Date: 22/11/24	
System: BudgetBliss Events	
Objective: Allow document sharing	***Test ID:*** TC-17
Version: 1	***Test Type:*** System Testing
Input: **Upload:** checklist.pdf	
Expected Result: File uploaded and viewable	
Actual Result: **Passed**	

Chapter 6

6. Software Deployment

6.1. Installation / Deployment Process Description

Step 1: Open GitHub Link

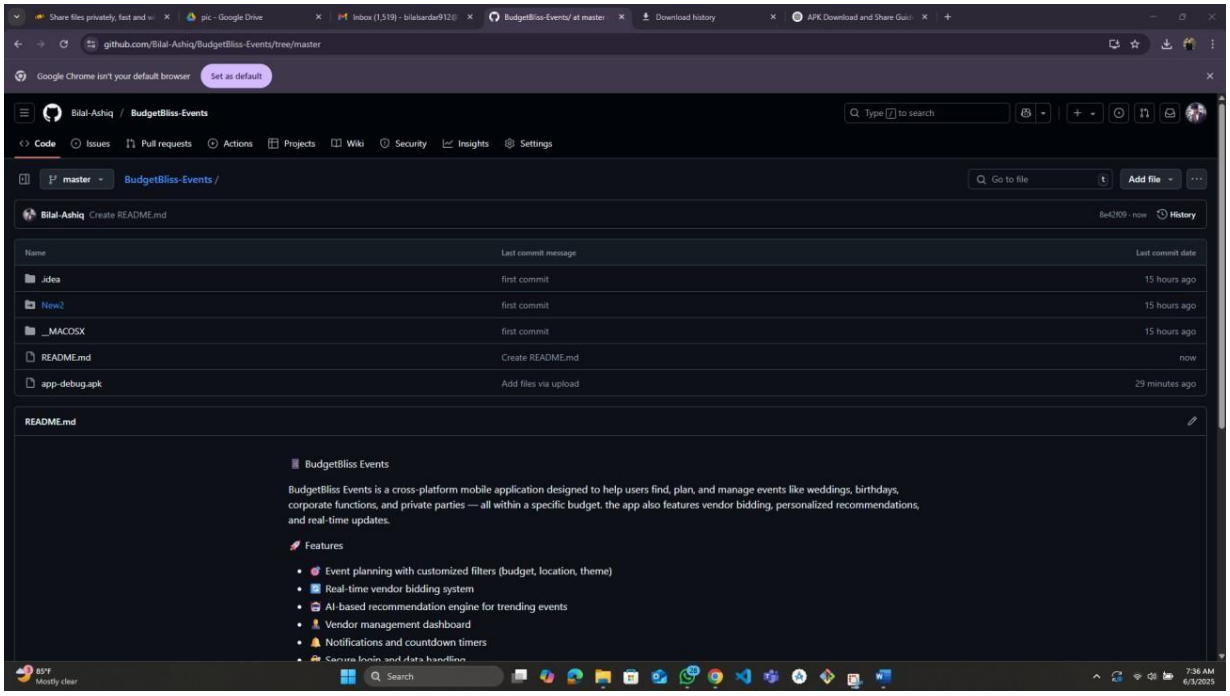
Description:

Open your browser and go to the GitHub repository for BudgetBliss. Link: https://github.com/Bilal-Ashiq/BudgetBliss-Events

Step 2: Locate the APK File

Description:

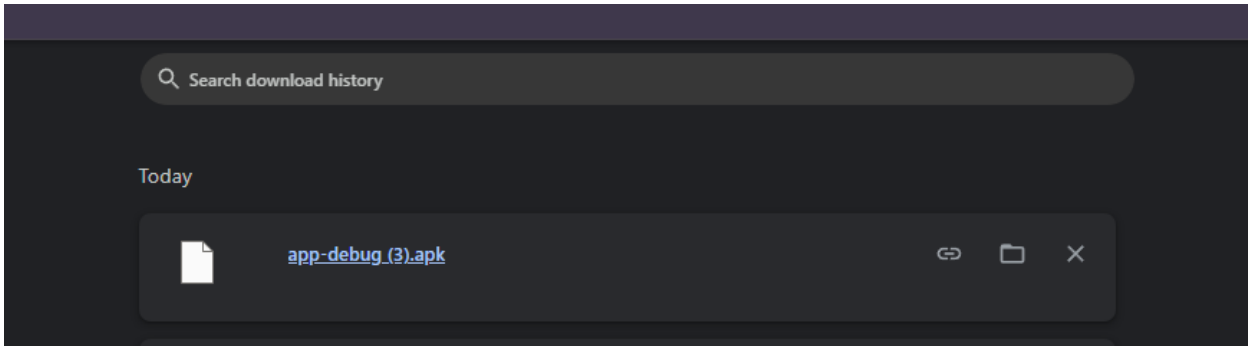
Scroll through the repository files. You'll see a .apk file such as App-debug.apk. Tip: Use Ctrl+F to search for .apk.



Step 3: Download APK File

Description:

Click on the APK file. GitHub will show a page with a "Download" button. Tap **Download** to begin downloading the APK file to your device.



Step 4: Share the APK File

Description:

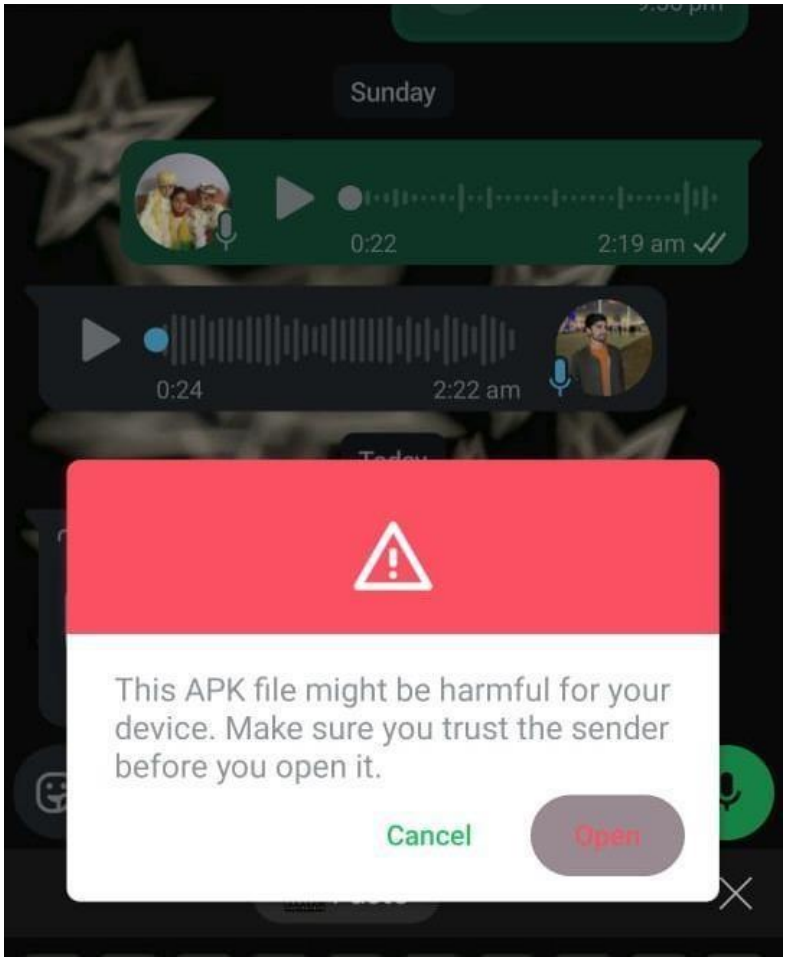
After the APK is downloaded:

1. Open your **File Manager**.
2. Navigate to **Downloads**.
3. Locate BudgetBliss.apk.

Step 5: Receiver Downloads the File

Description:

On the recipient's side, they will receive the APK file in WhatsApp or another app. They should **tap on the file**, then choose **Download** to save it locally.

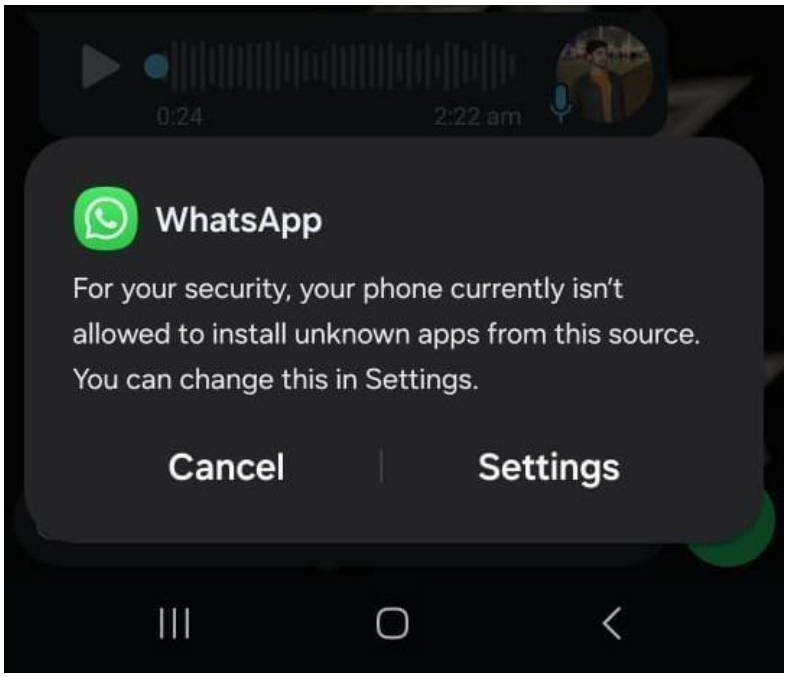


Step 6: Enable Installation from Unknown Sources

Description:

On the recipient's phone:

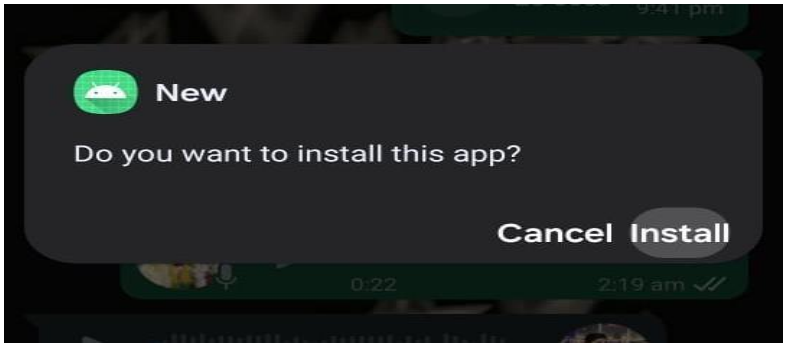
1. Go to **Settings > Security**.
2. Enable **"Install unknown apps"** for the app they used to open the APK (e.g., WhatsApp or File Manager).

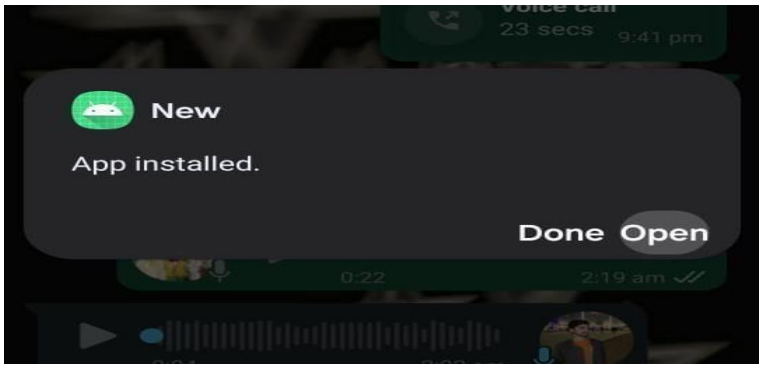


Step 7: Install the APK

Description:

Now, simply tap on the downloaded BudgetBliss.apk file. Click **Install** and wait for the app to be installed on the device.


New
Do you want to install this app?
Cancel
Install


New
App installed.
Done
Open


5:48
48%
WELCOME TO
BUDGET
BLISS

References

[1] D. Witt, "Key success factors for managing special events: The case of wedding tourism.," (Doctoral dissertation, North-West University)., 2006.

[2] Y. B. a. I. R. K. Kusuma, "Photography Business Opportunities in the 4.0 Industrial Revolution.," in *PROCEEDINGS 3RD ICONIDS INNOVATION*, 2022.

[3] *https://www.dawatpakistan.com/* [Online].

[4] *https://evento.com.pk/* [Online].

[5] *https://play.google.com/store/apps/details?id=net.mazein.weddingcounter*.

[6] "https://play.google.com/store/apps/details?id=com.bridebook.mobileapp," [Online].

[7] "https://play.google.com/store/apps/details?id=app.mywed.android," [Online].

Plagiarism Report

Report

ORIGINALITY REPORT

6%	5%	0%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	vdocument.in Internet Source	1%
2	www.coursehero.com Internet Source	1%
3	users.csc.calpoly.edu Internet Source	1%
4	Submitted to Regis University Student Paper	1%
5	123dok.com Internet Source	<1%
6	Submitted to Higher Education Commission Pakistan Student Paper	<1%
7	www.kcsfoundation.org Internet Source	<1%
8	Submitted to University of Westminster Student Paper	<1%
9	dokumen.tips Internet Source	<1%

Report Approval Certificate

The report of the project, "BudgetBliss Events" has been approved based on the following evaluation guideline.

Project Evaluation Guidelines

Artifacts Guidelines	
Analysis and Design artifacts are syntactically correct (use-case model, SSDs, domain model, class diagram, SDs, ERDs, Flow charts, Activity Diagram, DFDs)	
Consistency and traceability have been maintained among different artifacts	
General Guidelines	
Formatting (font style, indentation) is according to the FYP template and consistent throughout the document	
Captions are added to all the figures and tables. Figure captions must be placed below each figure, and table captions must be provided above the table	
Each figure or table is followed by some text describing what it represents	

Dr. Farah Haneef.

Farah Haneef.

Name & Signature

(Supervisor)